

ResUp

GROUP 1: Tanvi Jambhale, Jessica Fowler, Li Yu Tang

March 16 2026

Table of Contents

Introduction	2
--------------------	---

Datasets.....	2
---------------	---

Databases.....	3
----------------	---

Task 0.....	7
-------------	---

Task 1.....	11
-------------	----

Task 2.....	15
-------------	----

Task 3.....	21
-------------	----

Task 4.....	27
-------------	----

Task 5	35
--------------	----

Limitations and Future Directions	49
---	----

Introduction

Nowadays, even getting an interview can feel like reaching a star. To improve their chances, students often try to strengthen their resumes by improving their academic performance, developing technical skills, or gaining practical experience through internships and projects.

The goal is to build a resume strong enough for a recruiter or HR to look at it and think, “**OH YES, this is the one.**” However, deciding which area to focus on can be difficult without data-driven insights.

This application analyzes student profile data to determine which factors — academics, skills, or experience — have the greatest influence on placement outcomes across different majors, helping students understand what they should improve to increase their chances of landing an interview.

The GitHub repository contains all project implementations organized by task folders, where each folder corresponds to one database system used in the project (Task0–Task5).

<https://github.com/tjambhale/202-Final-Project>

The GitHub repository contains all project components organized by task folders. The Resume Data folder contains 20 resumes generated using an LLM (approved by the TA) to simulate realistic student profiles, and three additional versions were created for each resume to support testing in later tasks. The data folder includes the four Kaggle datasets used in the analysis. The masterResume folder contains longer versions of the 20 resumes with additional details such as extracurricular activities and extended experiences. Each task folder (task0–task5) contains the corresponding scripts, JSON files, or Jupyter notebooks used to process the datasets and generate the results for that part of the project.

Datasets

Kaggle Datasets

We integrated multiple Kaggle datasets to cover complementary aspects of student placement (academics, skills, and experience). Each dataset contributes different columns and distributions, which helps reduce reliance on any single schema.

1. College Student Placement Factors

- Source: <https://www.kaggle.com/datasets/sahilislam007/college-student-placement-factors-dataset?>
- This dataset contains strong placement-related predictors that directly match our feature design for **academics + experience + skills/communication**. Typical columns include CGPA/GPA-like measures, internships, academic performance indicators, and communication-related fields. We use it as a primary reference for building and validating the **academic/skill/experience** scoring logic.

2. Job Placement

- Source : <https://www.kaggle.com/datasets/ahsan81/job-placement-dataset?>
- This dataset commonly includes categorical education attributes and work experience fields. It improves our ability to compare **experience vs. academics** and supports more grouping/bucketing across different profile structures.

3. Student Job Profile

- Source : <https://www.kaggle.com/datasets/yuvjeetarora/student-job-profile>
- This dataset helps define and validate a **skill scoring** component (skill-set indicators that can be mapped to a normalized skill score/bucket). It supports our goal of producing interpretable recommendations.

4. Student Placement Prediction 2026

- Source : <https://www.kaggle.com/datasets/sehaj1104/student-placement-prediction-dataset-2026?>

- These datasets add additional diversity in majors, education attributes, and placement outcomes. We use them to support cross-checking placement trends and to ensure our derived features generalize beyond a single dataset.

Databases

We frame our system as a multi-database advising workflow, each database is chosen for a specific data shape, query pattern, and each task builds on the outputs of earlier tasks. Our pipeline uses four database systems: PostgreSQL, MongoDB, ChromaDB, and Neo4j.

PostgreSQL : Relational Database

- Used in Task 0 and 1 which require structured analytics over placement-related attributes (academics, skills, experience) and reproducible aggregation. A relational database is the best fit for schema-enforced columns, SQL-based filtering and grouping, and computing statistics.
- Operations:
 - clean and normalize inconsistent raw fields into standardized column
 - aggregate results to compute placement rates per major and per bucket
 - filter the data to select a small demo of 20 students using Task 1 constraints.
- PostgreSQL acts as our foundational analytical reference, it provides interpretable statistics and baseline scoring logic that later tasks reuse

MongoDB : NoSQL, Document Database

- Used in Tasks 2 and 3
- MongoDB is a best fit for resume data because resumes are nested, variable-length documents and different students may have different fields. A document database lets us store each resume version as a JSON-like document without forcing a rigid table schema.
- We store multiple resume versions per student (original + versions) as nested documents, enabling version-by-version comparisons and personalized resume generation.
- Operations:
 - Insert a new resume document when a student uploads an updated resume, while keeping older versions for history and revert.
 - Fetching latest versions by using `sort + limit` (by `timestamp`) to retrieve the two most recent resume versions for a student.
 - Nested field queries & projections for query and project nested arrays/fields to compare changes at the section level.
 - Aggregation pipelines used were `$match`, `$unwind`, `$group`, and `$project` to compute structured outputs such as added/removed skills, project changes, and category-level progress summaries.
- MongoDB supports our resume evolution features, a version timeline, a readable change log (added/removed/modified), and a tailored one-page resume output by selecting the most relevant bullets for a target role/keyword set.

ChromaDB : Vector Database

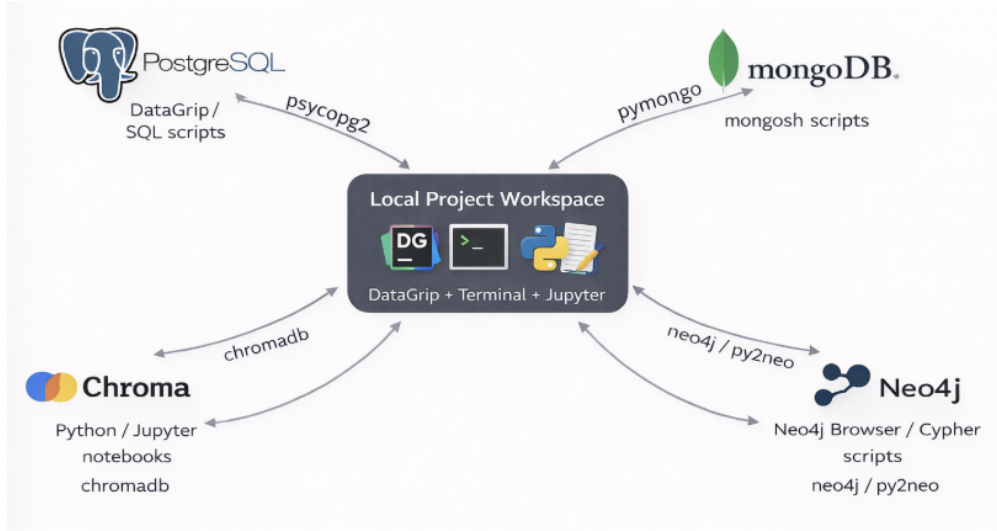
- Used in Task 4
- ChromaDB is used for semantic similarity search over student profiles/resumes. Unlike SQL filters (exact matches) or MongoDB document lookups, a vector database lets us retrieve the most similar students to a newly uploaded resume by comparing embedding vectors (approximate nearest neighbors).
- Each student profile is converted into a single text representation. We then compute an embedding for that text and store it in ChromaDB along with metadata.
- Operations:
 - Encode each profile/resume text using a sentence-transformer model to produce normalized vectors.
 - Store embeddings in a persistent local Chroma collection (so we do not recompute embeddings every run).
 - Given a query resume, compute its embedding and retrieve top- k most similar students using cosine distance.
 - Optionally filter candidates by attributes

Neo4j : Graph Database

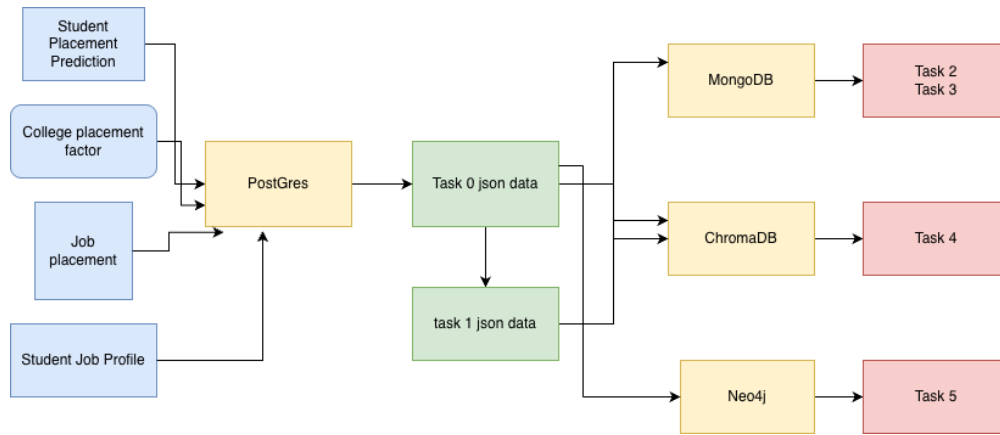
- Used in Task 5
- Neo4j is used to represent our data as a graph (nodes + relationships) so we can model connections between students and their attributes in a way that is difficult to express naturally in a purely relational schema.
- Nodes, Edges, and properties are stored
- Operations:
 - Find students connected to the same major and overlapping skills.
 - Retrieve students who share the most neighbors
 - Identify clusters of students within a major based on shared skill nodes, and summarize outcomes (placed vs not placed) per cluster.
 - Explain why two students are similar by showing the connecting path
- Neo4j makes it easy to answer relationship-centric questions like: “Which skills connect the highest-placement students in my major?” or “Who is in my major cluster and what do they have that I am missing?” This supports explainability by returning both the neighbor students and the shared graph evidence (nodes/edges) that connect them.

Overall Connection:

How the Databases are connected:



How the Datasets, Tasks, and Databases are connected:



Our system is connected through a staged multi-database workflow inside GitHub repository. Rather than having all databases query each other directly in real time, each task produces a structured output artifact—such as a JSON file, CSV file, PostgreSQL table/view, MongoDB collection, Chroma collection, or Neo4j graph—that becomes the input to a later step. The main integration layer is the local project workspace, where DataGrip SQL scripts, mongosh scripts, and Python/Jupyter code load prior outputs and generate new ones. This makes the system reproducible, since every stage can be rerun from saved files and clearly documented scripts.

The workflow begins in **Task 0 (PostgreSQL)**, where the raw placement datasets are cleaned and standardized into a common student-level structured fields. This serves as the project’s structured analytical foundation. In **Task 1 (PostgreSQL)**, the 20 original resumes are converted into comparable structured features, and Task 0’s major-level lift logic is used to assign each student a focus area and recommendation. Next, **Task 2 (MongoDB)** uses the resume version data from the Resume Data folder and combines it with task1 profiles. Its setup script merges the original, v1, v2, and v3 resume collections into a single resume versions collection, then compares versions and stores change logs. Because it looks up each student’s focus area from Task 1, it can determine whether the student actually improved in the area Task 1 recommended. **Task 3 (MongoDB)** is a related but separate resume-processing branch. It uses the master resumes collection to generate a tailored one-page resume for a chosen target role. In the current code, this task is guided by a fixed list of placement-related keywords inspired by the project’s structured categories.

Task 4 (ChromaDB + PostgreSQL) is where semantic similarity is added. The Python script loads the original resumes, Task 1’s structured resume profiles, and Task 0’s structured output. It builds Chroma collections for resume-text embeddings and score-profile embeddings, while also storing placement prediction values in PostgreSQL. ChromaDB is then used to retrieve the top 5 semantically similar resume peers, and PostgreSQL is used to retrieve stronger comparison students and compute improvement direction. This is a good example of two database systems working together in the same task, Chroma handles similarity retrieval, while PostgreSQL handles structured comparison and ranking. Finally, **Task 5 (Neo4j)** loads the final structured output from Task 0 into a graph of students, majors, bucket levels, and outcomes. This graph is used to show relationship-based patterns. In the current implementation, Task 5 is fed directly from Task 0’s final JSON, so it is best described as a graph-based explanation layer built on top of the structured foundation.

Overall, the system is best understood as a left-to-right workflow: PostgreSQL builds the structured base and recommendations; MongoDB handles resume document evolution and tailored-resume generation; ChromaDB adds semantic similarity search; and Neo4j visualizes relationship patterns from the structured placement data. The databases are connected by shared student keys, reused scoring logic, mapped categories, and exported artifacts stored in the repository.

Task 0

Goal:

Want to find what is most important for your major? Academics, skills, or experience? To start off, we need data. So before anything gets started, we will be building the foundation for all future tasks that will show insight to your resume. Datasets will be combined, bucket scores will be calculated and sorted, which will help compute lift scores. These computations will allow you to group information by major and see what the most important bucket is for you!

Main Files:

- College Student Placement Factors.csv
- Job Placement.csv
- Student Job Profile.csv
- Student Placement Prediction.csv (keeps the first 2,000 rows)

As two of our datasets contain many rows, we decided to limit the number of rows to reduce the push time required to GitHub. We decided to use **College Student Placement Factors.csv** and the main cohort to build our foundational data, thus used all 10,000 rows present in the dataset. Then decided to use the first 2,000 rows in **Student Placement Prediction.csv** and treating it as the secondary cohort. We decided to keep all rows for **Student Job Profile.csv** and **Job Placement.csv** as they do not contain many rows at all, and they would serve as supporting data as well.

How it Works:

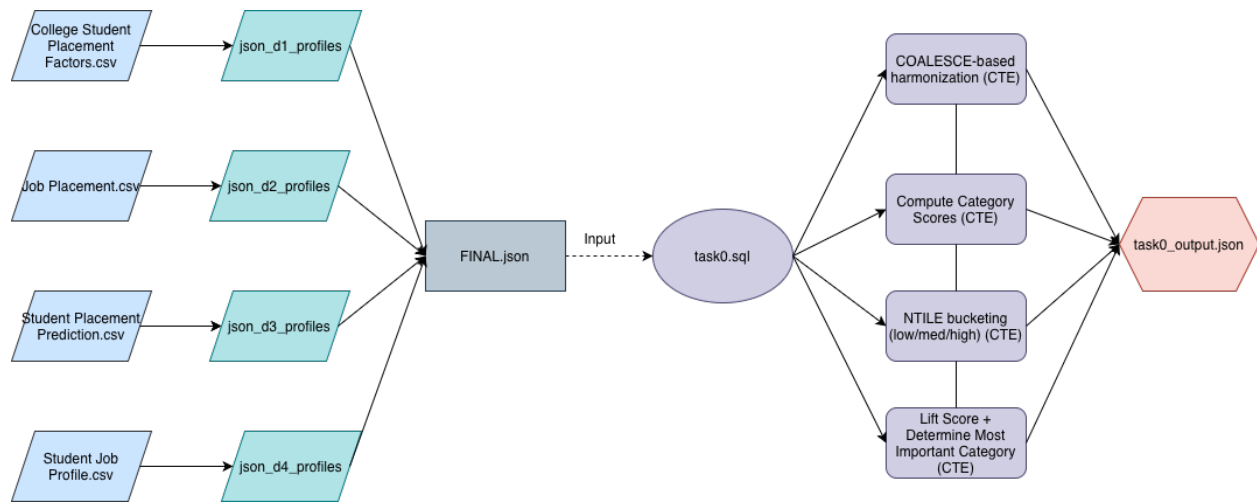


Figure 1: Overview of Task 0 pipeline

Step 1: Load Raw Data and Conduct Dataset-Specific JSON Normalization

Four raw CSV datasets were imported into a PostgreSQL database under the `proj` schema. Each dataset contain student academic, skill, and experience attributes with varying column structures, naming conventions, and placement outcome formats. No modifications were done to the datasets and imported as-is to preserve the original data.

Each dataset was then individually processed through unique SQL scripts that transform into a standardized JSONB structure. Each student was assigned a unique key, then organized into consistent nested categories: `academics`, `skills`, `experience`, and `outcome`. In these independent queries, dataset-specific inconsistencies such as differing column names, binary yes/no fields, and missing placement labels were handled. The goal was to produce a uniform schema across all four datasets regardless of their original structure.

Step 2: Combine All 4 Datasets Together

To form our foundational data that will be the backbone of all future features, we need to combine the data.

The four normalized JSONB tables are combined into a single table using a source-prefixed global key to uniqueness across datasets. This output table is exported as `FINAL.json`, which will serve as the single input for all future analysis in this task.

```
{
  "profile": {"raw": {"iq": 101, "cgpa": 7.25, "placement": "No", "college_id": "CLG0001", "prev_sem_result": 7.25,
    "projects_completed": 5, "academic_performance": 9, "communication_skills": 4, "internship_experience": "Yes",
    "extra_curricular_score": 10}, "skills": {"communication_skills": 4, "extra_curricular_score": 10}, "source":
    "college_student_placement_factors", "outcome": {"placed": 0}, "academics": {"iq": 101, "cgpa": 7.25, "prev_sem_result": 7.25,
    "academic_performance": 9}, "experience": {"projects_completed": 5, "internship_experience": "Yes"}, "major_group": "CLG0001",
    "student_key": "d1_1"
  },
}
```

Figure 2: Example 1 of combined data

```
{
  "profile": {"raw": {"rn": 1080, "age": 18, "cgpa": 6.59, "branch": "Mechanical", "gender": "Female", "backlogs": "0",
    "student_id": 1080, "sleep_hours": "5.6", "college_tier": "Tier 2", "github_repos": "1", "aptitude_score": 49.3379341753014,
    "projects_count": 2, "leadership_score": "68.26970873213915", "placement_status": "Not Placed", "internships_count": 1,
    "coding_skill_score": 61.8041460827123, "salary_package_lpa": "0.0", "study_hours_per_day": "2.0", "certifications_count": 2,
    "linkedin_connections": "189", "mock_interview_score": "72.97022927025426", "volunteer_experience": "Yes", "attendance_percentage":
    "91.43488697882417", "extracurricular_score": "80.92323288032077", "hackathons_participated": "0", "logical_reasoning_score":
    85.22247453043299, "communication_skill_score": 67.17005030667778}, "skills": {"aptitude_score": 49.3379341753014,
    "coding_skill_score": 61.8041460827123, "mock_interview_score": "72.97022927025426", "logical_reasoning_score": 85.22247453043299,
    "communication_skill_score": 67.17005030667778}, "source": "student_placement_prediction", "outcome": {"placed": 0,
    "placement_status": "Not Placed", "salary_package_lpa": "0.0"}, "academics": {"cgpa": 6.59, "backlogs": "0", "study_hours_per_day":
    "2.0", "attendance_percentage": "91.43488697882417"}, "experience": {"github_repos": "1", "projects_count": 2, "leadership_score":
    "68.26970873213915", "internships_count": 1, "certifications_count": 2, "volunteer_experience": "Yes", "extracurricular_score":
    "80.92323288032077", "hackathons_participated": "0"}, "major_group": {"primary": "Mechanical", "secondary": "Tier 2"},
    "student_key": "d3_1080", "demographics": {"age": 18, "gender": "Female", "sleep_hours": "5.6"}}
  },
}
```

Figure 3: Example 2 of combined data

Figure 1 and Figure 2 are example outputs that are within the combined `FINAL.json` file. The yellow box highlights the global `student_key`, and the coral pink are important grouped categories highlighted.

`FINAL.json` will be the input for `task0.sql`, which contains all code with computations for future analytics described in future steps.

Step 3: Score Computation

Using `FINAL.json`, three scores will be computed for each student: academic, skills, and experience. As each data sets contributes to different fields, a `COALESCE`-based harmonization strategy is used to extract relevant fields available for every student. Each contributing field is then normalized to a comparable score, then is averaged over only the non-null fields present. This ensures scores were not penalized for fields that simply did not exist in a given dataset.

Step 4: Bucketing

Once each student has their three numerical scores (academics, skills, and experience), `NTILE` is applied separately for each score dimension to assign every student a bucket label of Low, Medium, or High. Since `NTILE` divides the dataset into three equal-sized groups by rank, roughly one third of students fall into each

```

-- academics
COALESCE(
  (data->'profile'->'academics'->'cgpa')::numeric,
  (data->'profile'->'raw'->'cgpa')::numeric
) AS cgpa,

COALESCE(
  (data->'profile'->'raw'->'iq')::numeric,
  NULL
) AS iq,

COALESCE(
  (data->'profile'->'raw'->'prev_sem_result')::numeric,
  NULL
) AS prev_sem_result,

COALESCE(
  (data->'profile'->'raw'->'academic_performance')::numeric,
  NULL
) AS academic_performance,

COALESCE(
  (data->'profile'->'academics'->'attendance_percentage')::numeric,
  NULL
) AS attendance_percentage,

```

```

scores AS (
  SELECT
    *,
    CASE
      WHEN cgpa IS NULL
        AND iq IS NULL
        AND prev_sem_result IS NULL
        AND academic_performance IS NULL
        AND attendance_percentage IS NULL
      THEN NULL
      ELSE (
        COALESCE(cgpa, 0)
        + COALESCE(LEAST(GREATEST(iq / 14.0, 0), 10), 0)
        + COALESCE(prev_sem_result, 0)
        + COALESCE(academic_performance, 0)
        + COALESCE(attendance_percentage / 10.0, 0)
      )
    /
    NULLIF(
      (CASE WHEN cgpa IS NOT NULL THEN 1 ELSE 0 END) +
      (CASE WHEN iq IS NOT NULL THEN 1 ELSE 0 END) +
      (CASE WHEN prev_sem_result IS NOT NULL THEN 1 ELSE 0 END) +
      (CASE WHEN academic_performance IS NOT NULL THEN 1 ELSE 0 END) +
      (CASE WHEN attendance_percentage IS NOT NULL THEN 1 ELSE 0 END),
      0
    )
  )
END AS academic_score,

```

Figure 4: Left: COALESCE-based harmonization for the academic bucket. Right: Computation of academic score

bucket per category. Bucketing is performed twice — once globally across all students, and once partitioned within each major group — allowing for both cross-dataset comparisons and major-specific analysis. All bucket assignments across the three categories are independent of each other. These bucket boundaries are not fixed values; they adjust automatically based on the score distribution of the dataset.

```

ranked AS (
  SELECT
    *,
    CASE
      WHEN academic_score IS NOT NULL
      THEN NTILE(3) OVER (ORDER BY academic_score NULLS LAST)
    END AS academic_tile_all,

    CASE
      WHEN skill_score IS NOT NULL
      THEN NTILE(3) OVER (ORDER BY skill_score NULLS LAST)
    END AS skill_tile_all,

    CASE
      WHEN experience_score IS NOT NULL
      THEN NTILE(3) OVER (ORDER BY experience_score NULLS LAST)
    END AS experience_tile_all,

    CASE
      WHEN major IS NOT NULL AND academic_score IS NOT NULL
      THEN NTILE(3) OVER (PARTITION BY major ORDER BY academic_score NULLS LAST)
    END AS academic_tile_major,

    CASE
      WHEN major IS NOT NULL AND skill_score IS NOT NULL
      THEN NTILE(3) OVER (PARTITION BY major ORDER BY skill_score NULLS LAST)
    END AS skill_tile_major,

    CASE
      WHEN major IS NOT NULL AND experience_score IS NOT NULL
      THEN NTILE(3) OVER (PARTITION BY major ORDER BY experience_score NULLS LAST)
    END AS experience_tile_major
  FROM scores
)

```

```

CASE
  WHEN r.academic_score IS NULL THEN NULL
  WHEN r.academic_tile_all = 1 THEN 'Low'
  WHEN r.academic_tile_all = 2 THEN 'Medium'
  WHEN r.academic_tile_all = 3 THEN 'High'
END AS academic_bucket,

CASE
  WHEN r.skill_score IS NULL THEN NULL
  WHEN r.skill_tile_all = 1 THEN 'Low'
  WHEN r.skill_tile_all = 2 THEN 'Medium'
  WHEN r.skill_tile_all = 3 THEN 'High'
END AS skill_bucket,

CASE
  WHEN r.experience_score IS NULL THEN NULL
  WHEN r.experience_tile_all = 1 THEN 'Low'
  WHEN r.experience_tile_all = 2 THEN 'Medium'
  WHEN r.experience_tile_all = 3 THEN 'High'
END AS experience_bucket,

```

Figure 5: Left: NTILE bucketing globally and by major. Right: Label conversion

Step 5: Compute the Lift Scores and Determine Most Important Category

Lift scores are a measure of how much a category (academics, skills, or experience) actually affect your chances of getting placed within your major. For each major group, placement rates were computed separately for the top bucket (high) and bottom bucket (low). The difference between these rates is the lift. The category with the highest lift is named as the most important category for that major group. This indicates which

category (academics, skills, or experience) is the most important predictive outcome of placement outcomes.

```
major_importance AS (
  SELECT
    major,
    placement_rate_high_academic,
    placement_rate_low_academic,
    placement_rate_high_academic - placement_rate_low_academic AS academic_lift,
    placement_rate_high_skill,
    placement_rate_low_skill,
    placement_rate_high_skill - placement_rate_low_skill AS skill_lift,
    placement_rate_high_experience,
    placement_rate_low_experience,
    placement_rate_high_experience - placement_rate_low_experience AS experience_lift,
    CASE
      WHEN (placement_rate_high_academic - placement_rate_low_academic) >=
        (placement_rate_high_skill - placement_rate_low_skill)
      AND (placement_rate_high_academic - placement_rate_low_academic) >=
        (placement_rate_high_experience - placement_rate_low_experience)
      THEN 'Academics'
      WHEN (placement_rate_high_skill - placement_rate_low_skill) >=
        (placement_rate_high_academic - placement_rate_low_academic)
      AND (placement_rate_high_skill - placement_rate_low_skill) >=
        (placement_rate_high_experience - placement_rate_low_experience)
      THEN 'Skills'
      ELSE 'Experience'
    END AS most_important_category
  FROM major_rates
```

Figure 6: How to calculate lift and determine most important category by major

Running and Testing:

Output:

student_key	major	academic_score	skill_score	experience_score	academic_bucket
d3_100	CSE	6.8344243163550835	16.2139820699943811	1.3333333333333333	Medium
d3_1002	CSE	7.0910250225071225	18.9986033699044158	2.6666666666666667	Medium
d3_1003	CSE	8.261647385163565	21.1203023193035016		2 High
d3_1005	CSE	8.1825889201508005	23.8122314037060058	3.6666666666666667	High
d3_1008	CSE	8.6945444499359285	19.828296012363904	1.6666666666666667	High

skill_bucket	experience_bucket	placed	placement_rate_high_academic	placement_rate_low_academic
High	Medium	0	0.54782608695652173913	0.6060606060606061
High	High	0	0.54782608695652173913	0.6060606060606061
High	Medium	0	0.54782608695652173913	0.6060606060606061
High	High	1	0.54782608695652173913	0.6060606060606061
High	Medium	1	0.54782608695652173913	0.6060606060606061

academic_lift	placement_rate_high_skill	placement_rate_low_skill	skill_lift
-0.05823451910408432148	0.63913043478260869565		0.56709956709956709957
-0.05823451910408432148	0.63913043478260869565		0.56709956709956709957
-0.05823451910408432148	0.63913043478260869565		0.56709956709956709957
-0.05823451910408432148	0.63913043478260869565		0.56709956709956709957
-0.05823451910408432148	0.63913043478260869565		0.56709956709956709957

placement_rate_high_experience	placement_rate_low_experience	experience_lift	most_important_category
0.63043478260869565217	0.52813852813852813853	0.10229625447016751364	Experience
0.63043478260869565217	0.52813852813852813853	0.10229625447016751364	Experience
0.63043478260869565217	0.52813852813852813853	0.10229625447016751364	Experience
0.63043478260869565217	0.52813852813852813853	0.10229625447016751364	Experience
0.63043478260869565217	0.52813852813852813853	0.10229625447016751364	Experience

Figure 7: Example output for Task 0 (all rows from same student key, results are all from same rows).

Task 1

Goal:

You and 20 of your friends have completed assembling your resumes, however your resumes do not tell you where you each stand among your peers. This feature will help score every resume across three categories and compute scores for them which will be ranked: academics, skills, and experience. You will now see low/med/high buckets for these three categories. You will now see output that will recommend what you should work on next to maximize your hiring odds!

Main Files:

- `resumes-original.json` (in ResmumeData folder)
 - It is to be noted that these resumes are generated using LLM's
- `task0_final_output.json`

How it Works:

Step 1: Upload Resumes into PostgreSQL

Just like Task0, resumes are stored in JSON format. These resumes are unnested as stored into PostgreSQL to be evaluated.

Step 2: Compute Scores and Lifts

Also like Task 0, category scores, and category buckets will be computed across all 20 resumes uploaded using the same methods as in our foundational database produced in Task 0.

- Category scores are calculated using weights to take into account more important factors of a resume that have larger impact on potential job placement
- Category buckets are once again evaluated using NTILE

```
0.60 * LEAST((cgpa / 4.0) * 10.0, 10) +  
0.25 * LEAST(course_count * 2.0, 10) +  
0.15 * LEAST(honor_count * 3.0, 10)  
:::numeric, 4) AS academic_score,
```

Figure 8: Example calculation of academic score in Task 1

Step 3: Mapping Majors in Resumes to Foundational Database

As students are most likely to have different working of majors to Task 0, major names are mapped with each other, then joined against `task0_final_output` table to retrieve the average academic, skills, and experience lift scores for that major group.


```

CASE b.major
  WHEN 'Computer Science' THEN 'CSE'
  WHEN 'Computer Science: Game Design' THEN 'CSE'
  WHEN 'Computer Engineering' THEN 'CSE'
  WHEN 'Software Engineering' THEN 'Software Engineer'
  WHEN 'Data Science' THEN 'IT'
  WHEN 'Information Systems' THEN 'IT'
  WHEN 'Electrical Engineering' THEN 'EEE'
  WHEN 'Mechanical Engineering' THEN 'Mechanical'
  WHEN 'Economics' THEN 'Comm&Mgmt'
  WHEN 'Mathematics' THEN 'Sci&Tech'
  WHEN 'Statistics' THEN 'Sci&Tech'
  WHEN 'Design' THEN 'UI/UX'
  WHEN 'Cognitive Science (HCI)' THEN 'UI/UX'
  WHEN 'Biomedical Engineering' THEN 'ECE'
  ELSE b.major
END AS task0_major

```

Figure 9: Demonstration of major name mapping

Step 4: Determine Focus Area

For each student, a recommendation string will be generated. The focus area is determined by following a priority logic:

- GPA override: if CGPA $\leq$ 3.0 and academic bucket is low, the focus area is forced to be academics regardless of lift values
 - A 3.0 GPA is usually a hard filter for most employers!
- Otherwise, the category with the highest lift calculated from Task 0 is selected as the focus area
 - Experience wins ties with skills
 - Skills wins ties over academics

The recommendation string is now produced based on the focus bucket:

- URGENT: focus bucket is low (weakest area and highest placement lift), needs immediate attention
- GROW: focus bucket is medium (strong leverage opportunity to push from medium to high)
- MAINTAIN: focus bucket is already high, sustain and shift attention to the next area

```

-- RECOMMENDATION string
CASE
  -- GPA override recommendation
  WHEN cgpa < 3.0 AND academic_bucket = 'Low' THEN
    'URGENT - GPA is ' || ROUND(cgpa,2)::text || ' (below 3.0) and Academic score is Low. ' ||
    'Raise your GPA first - this is a hard filter for most employers. ' ||
    'Attend office hours, retake low-grade courses, and reduce extracurriculars temporarily.'

  WHEN focus_bucket = 'Low' THEN
    'URGENT - ' || focus_area || ' is your weakest area AND has the highest placement lift for ' ||
    major || ' majors (lift: ' ||
    ROUND(CASE focus_area
      WHEN 'Experience' THEN COALESCE(experience_lift,0)
      WHEN 'Skills' THEN COALESCE(skill_lift,0)
      ELSE COALESCE(academic_lift,0)
    END::numeric, 3)::text || '). ' ||

  CASE focus_area
    WHEN 'Experience' THEN 'Get an internship and complete 1-2 more strong projects.'
    WHEN 'Skills' THEN 'Add 2-3 in-demand technical skills and build a portfolio.'
    ELSE 'Raise GPA above 3.7 and pursue honors or research.'
  END

```

Figure 10: Case to produce URGENT recommendation string.

```

WHEN focus_bucket = 'Medium' THEN
  'GROW - Push ' || focus_area || ' from Medium to High. ' ||
  'This is the highest-leverage move for ' || major || ' placement (lift: ' ||
  ROUND(CASE focus_area
    WHEN 'Experience' THEN COALESCE(experience_lift,0)
    WHEN 'Skills'      THEN COALESCE(skill_lift,0)
    ELSE                COALESCE(academic_lift,0)
  END::numeric, 3)::text || '). ' ||
  CASE focus_area
    WHEN 'Experience' THEN 'Take on a leadership role and add one more substantial project.'
    WHEN 'Skills'      THEN 'Deepen expertise in your top skills and earn a certification.'
    ELSE                'Target 3.7+ GPA and take advanced electives in your focus area.'
  END

ELSE
  'MAINTAIN - ' || focus_area || ' is already High. ' ||
  'Keep it strong and shift focus to your next weakest area to stay well-rounded for ' || major || ' roles.'
END AS recommendation

```

Figure 11: Case to produce GROW or MAINTAIN recommendation string.

Output:

u_0001	UC Davis	Computer Science	3.41	7.07	5.1	3.23	Medium	High	High	Experience	Experience
u_0002	UC Davis	Statistics	3.74	7.56	5.1	3.23	High	High	High		Experience
u_0003	San José State University	Software Engineering	3.18	6.72	5.1	3.23	Low	Low	High		Experience
u_0004	UC Irvine	Data Science	3.88	7.77	5.1	3.05	High	Low	Low	Experience	Experience
u_0005	UC San Diego	Cognitive Science (HCI)	3.29	6.89	5.1	3.23	Low	Low	Low		Experience
u_0006	UC Davis	Mechanical Engineering	3.05	6.53	5.1	3.05	Low	High	Low	Skills	Skills
u_0007	UC Davis	Computer Science	3.62	7.38	5.1	3.23	High	Low	Low	Experience	Experience
u_0008	UCLA	Electrical Engineering	3.53	7.25	5.1	3.23	Medium	Low	Low	Experience	Experience
u_0009	UC Berkeley	Economics	3.67	7.46	5.1	3.23	High	Low	Low		Experience
u_0010	UC Davis	Computer Engineering	3.22	6.78	5.1	3.23	Low	Medium	Medium	Experience	Experience
u_0011	California State University, Long Beach	Information Systems	3.56	7.29	5.1	3.23	Medium	Medium	Medium	Experience	Experience
u_0012	UC Santa Cruz	Computer Science: Game Design	3.12	6.63	5.1	3.23	Low	Medium	Medium	Experience	Experience
u_0013	UC Riverside	Computer Science	3.47	7.16	5.1	3.23	Medium	Medium	Medium	Experience	Experience
u_0014	UC Davis	Design	3.58	7.32	5.1	3.23	Medium	Medium	Medium		Experience
u_0015	UC Santa Barbara	Mathematics	3.81	7.67	5.1	3.23	High	Medium	Medium		Experience
u_0016	UC Davis	Data Science	3.33	6.95	5.1	3.23	Medium	Medium	Medium	Experience	Experience
u_0017	UC Davis	Computer Science	2.94	6.36	5.1	3.23	Low	High	High	Experience	Academics
u_0018	UC Merced	Computer Science	3.09	6.59	5.1	3.23	Low	High	High	Experience	Experience
u_0019	UC Davis	Biomedical Engineering	3.44	7.11	5.1	3.05	Medium	Low	Low	Experience	Experience
u_0020	UC Davis	Computer Science	3.95	7.88	6.15	3.23	High	High	High	Experience	Experience

Figure 12: Left Side of Output Table

Column names (from left to right in Figure 11): student_key, college, major, cgpa, academic_score, skill_score, experience_score, academic_bucket, skill_bucket, experience_bucket, most_important_category, focus_area

MAINTAIN — Experience is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Computer Science roles.
MAINTAIN — Experience is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Statistics roles.
MAINTAIN — Experience is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Software Engineering roles.
URGENT — Experience is your weakest area AND has the highest placement lift for Data Science majors (lift: 0.181). Get an internship and complete 1-2 more strong projects.
URGENT — Experience is your weakest area AND has the highest placement lift for Cognitive Science (HCI) majors (lift: 0.000). Get an internship and complete 1-2 more strong projects.
MAINTAIN — Skills is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Mechanical Engineering roles.
URGENT — Experience is your weakest area AND has the highest placement lift for Computer Science majors (lift: 0.107). Get an internship and complete 1-2 more strong projects.
URGENT — Experience is your weakest area AND has the highest placement lift for Electrical Engineering majors (lift: 0.209). Get an internship and complete 1-2 more strong projects.
URGENT — Experience is your weakest area AND has the highest placement lift for Economics majors (lift: 0.000). Get an internship and complete 1-2 more strong projects.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Computer Engineering placement (lift: 0.107). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Information Systems placement (lift: 0.181). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Computer Science: Game Design placement (lift: 0.107). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Computer Science placement (lift: 0.107). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Design placement (lift: 0.000). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Mathematics placement (lift: 0.000). Take on a leadership role and add one more substantial project.
GROW — Push Experience from Medium to High. This is the highest-leverage move for Data Science placement (lift: 0.181). Take on a leadership role and add one more substantial project.
URGENT — GPA is 2.94 (below 3.0) and Academic score is Low. Raise your GPA first — this is a hard filter for most employers. Attend office hours, retake low-grade courses, and reduce extracurriculars temporarily.
MAINTAIN — Experience is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Computer Science roles.
URGENT — Experience is your weakest area AND has the highest placement lift for Biomedical Engineering majors (lift: 0.198). Get an internship and complete 1-2 more strong projects.
MAINTAIN — Experience is already High. Keep it strong and shift focus to your next weakest area to stay well-rounded for Computer Science roles.

Figure 13: Right Side of Output Table

This is the last column that contains the recommendation strings for all 20 resumes.

Task 2

Goal: You've just finished updating your resume based on feedback from Task 1, but did your changes actually move the needle? You added some skills and a new project, but you're not sure if you improved in the area that matters most for your major. And what exactly changed between your last version and this one? Resume Tracker can help you see precisely what changed in your resume over time, score each version, and tell you whether you improved where it counts.

Main Files:

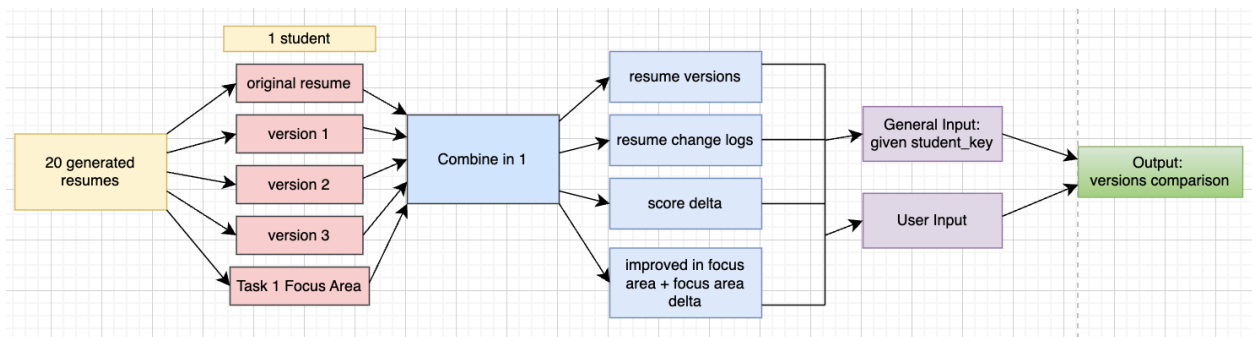
Inside the task2 folder:

- `task2_mongodb_setup.js` – run this before because it has the setup code as well as main code
- `user_play_versions.js` – additional feature, where user can input
- `student_id` folder – has the inputs to run for the 20 generated resumes in separated files
- `input` – what you can run in mongodb after the main code, made it easy to retrieve

Outside of task2 folder [in main repo]

- `ResumeData` folder – has all of the versions of the resume

How it Works:



Step 1: Resumes

A single `$unionWith` aggregation pipeline merges all four source collections into `resume_versions`. Each document is tagged with `version_label` ("original", "v1", "v2", "v3") and `version_order` (0-3) to enable deterministic sorting. A compound unique index on `{student_key, version_order}` prevents duplicates. The result is 80 documents total (20 students × 4 versions).

```

30     $unionWith: {
31       coll: "resumes_v1",
32       pipeline: [
33         {
34           $addFields: {
35             version_label: "v1",
36             version_order: 1,
37             source_file: "resumes-version1.json"
38           }
39         },
40         {
41           $project: {
42             _id: 0,
43             student_key: 1,
44             timestamp: { $toDate: "$timestamp" },
45             version_label: 1,
46             version_order: 1,
47             source_file: 1,
48             college: 1,
49             academics: 1,
50             skills: 1,
51             experience: 1
52           }
53         }
54       ]
55     }

```

Figure 14: From task2mongodbsetup.js

Step 2: Scoring

The same three scoring formulas from Task 1 are applied in JavaScript to every document in `resume_versions`. Scores are written back via `updateOne()`:

```

220 db.resume_versions.find().forEach(doc => {
221   db.resume_versions.updateOne(
222     { _id: doc._id },
223     {
224       $set: {
225         academic_score: academicScore(doc),
226         skill_score: skillScore(doc),
227         experience_score: experienceScore(doc)
228       }
229     }
230   );
231 });
232

```

Figure 15: From task2mongodbsetup.js

Step 3: Differences between Versions

For each student, consecutive resume versions are compared and a `resume_change_logs` document is created for each pair. The `compareResumeVersions(oldDoc, newDoc)` function computes structured differences across nested resume fields using normalized lowercase matching. It records what changed in skills, academics, and experience, along with the numeric score change in each category. The resulting `diff` object is then inserted into `resume_change_logs` together with metadata such as the student key, compared version labels, compared timestamps, and score-before / score-after values.

- **Skills:** technical and soft — added/removed string arrays
- **Academics:** CGPA delta, added/removed coursework names, added/removed award titles

- **Experience:** added/removed project names, work titles, and leadership activity names
- **Score delta:** numeric difference per category (academic, skill, experience)

Two comparison types are stored per student:

- **Step** (comparison_type: "step"): original→v1, v1→v2, v2→v3
- **Overall** (comparison_type: "overall"): original→v3

```

235  function compareResumeVersions(oldDoc, newDoc) {
236      const oldCourses = objNameSet(oldDoc?.academics?.relevant_coursework, "course");
237      const newCourses = objNameSet(newDoc?.academics?.relevant_coursework, "course");
238
239      const oldAwards = objNameSet(oldDoc?.academics?.honors_or_awards, "title");
240      const newAwards = objNameSet(newDoc?.academics?.honors_or_awards, "title");
241
242      const oldProjects = objNameSet(oldDoc?.experience?.projects, "name");
243      const newProjects = objNameSet(newDoc?.experience?.projects, "name");
244
245      const oldWork = objNameSet(oldDoc?.experience?.work, "title");
246      const newWork = objNameSet(newDoc?.experience?.work, "title");
247
248      const oldActs = objNameSet(oldDoc?.experience?.leadership_and_activities, "activity");
249      const newActs = objNameSet(newDoc?.experience?.leadership_and_activities, "activity");
250
251      return {
252          academics: {
253              cgpa_old: Number(oldDoc?.academics?.cgpa || 0),
254              cgpa_new: Number(newDoc?.academics?.cgpa || 0),
255              cgpa_delta: round4(
256                  Number(newDoc?.academics?.cgpa || 0) - Number(oldDoc?.academics?.cgpa || 0)
257              ),
258              coursework: setDiff(oldCourses, newCourses),
259              awards: setDiff(oldAwards, newAwards)
260          },
261          skills: {
262              technical: setDiff(oldDoc?.skills?.technical, newDoc?.skills?.technical),
263              soft: setDiff(oldDoc?.skills?.soft, newDoc?.skills?.soft)
264          },
265          experience: {
266              projects: setDiff(oldProjects, newProjects),
267              work_titles: setDiff(oldWork, newWork),
268              activities: setDiff(oldActs, newActs)
269          },
270          score_delta: {
271              academic: round4((newDoc?.academic_score || 0) - (oldDoc?.academic_score || 0)),
272              skill: round4((newDoc?.skill_score || 0) - (oldDoc?.skill_score || 0)),
273              experience: round4((newDoc?.experience_score || 0) - (oldDoc?.experience_score || 0))
274          }
275      };
276  }

```

Figure 16: From task2mongodbsetup.js

```

db.resume_change_logs.insertOne({
  student_key: studentKey,
  major: curr?.college?.major || profile?.major || null,
  comparison_type: "step",
  compared_versions: {
    from: prev.version_label,
    to: curr.version_label
  },
  compared_timestamps: {
    from: prev.timestamp,
    to: curr.timestamp
  },
  focus_area: focusArea,
  most_important_category: profile?.most_important_category || null,
  score_before: {
    academic: prev.academic_score,
    skill: prev.skill_score,
    experience: prev.experience_score
  },
  score_after: {
    academic: curr.academic_score,
    skill: curr.skill_score,
    experience: curr.experience_score
  },
  score_delta: diff.score_delta,
  improved_in_focus_area: focusDelta !== null ? focusDelta > 0 : null,
  focus_area_delta: focusDelta,
  changes: diff
});
}

```

Figure 17: From task2mongodbssetup.js

Step 4: Tie it back to Task1

Each change log document joins in the student's `focus_area` from `task1_profiles`. Two fields directly answer the core question:

- `improved_in_focus_area`: `true/false` — did the score go up in the recommended area?
- `focus_area_delta`: the exact score change in that category

Task 1 says “focus on Skills”, Task 2 confirms whether the student actually improved there.

Running and Testing:

From the 20 resumes generated, testing student with `student_key` [u_0001]:

```

use("proj");

db.resume_change_logs.find(
  { student_key: "u_0001" },
  {
    _id: 0,
    student_key: 1,
    comparison_type: 1,
    compared_versions: 1,
    focus_area: 1,
    focus_area_delta: 1,
    improved_in_focus_area: 1,
    score_before: 1,
    score_after: 1,
    score_delta: 1,
    "changes.academics": 1,
    "changes.skills": 1,
    "changes.experience": 1
  }
).sort({ "compared_timestamps.from": 1 });

```

Output:

```
[
  {
    student_key: 'u_0001',
    comparison_type: 'step',
    compared_versions: { from: 'original', to: 'v1' },
    focus_area: null,
    score_before: { academic: 7.065, skill: 5.1, experience: 3.225 },

    score_after: { academic: 7.64, skill: 6.15, experience: 4.025 },
    score_delta: { academic: 0.575, skill: 1.05, experience: 0.8 },
    improved_in_focus_area: null,
    focus_area_delta: null,
    changes: {
      academics: {
        cgpa_old: 3.41,
        cgpa_new: 3.46,
        cgpa_delta: 0.05,
        coursework: { added: [ 'project management' ], removed: [ ] }
      },
      awards: { added: [ ], removed: [ ] },
      skills: {
        technical: { added: [ 'unit testing' ], removed: [ ] },
        soft: { added: [ ], removed: [ ] }
      },
      experience: {
        projects: { added: [ 'resume change log script' ], removed: [ ] },
        work_titles: { added: [ ], removed: [ ] },
        activities: { added: [ ], removed: [ ] }
      }
    }
  },
],
```

(a) original to v1

```
{
  student_key: 'u_0001',
  comparison_type: 'overall',
  compared_versions: { from: 'original', to: 'v3' },
  focus_area: null,
  score_before: { academic: 7.065, skill: 5.1, experience: 3.225 },

  score_after: { academic: 8.29, skill: 8.35, experience: 4.825 },
  score_delta: { academic: 1.225, skill: 3.25, experience: 1.6 },
  improved_in_focus_area: null,
  focus_area_delta: null,
  changes: {
    academics: {
      cgpa_old: 3.41,
      cgpa_new: 3.56,
      cgpa_delta: 0.15,
      coursework: { added: [ 'project management', 'applied analytics' ], removed: [ ] },
      awards: { added: [ ], removed: [ ] }
    },
    skills: {
      technical: {
        added: [ 'unit testing', 'power bi', 'ci/cd', 'pandas' ],
        removed: [ ]
      },
      soft: { added: [ 'problem solving' ], removed: [ ] }
    },
    experience: {
      projects: {
        added: [ 'resume change log script', 'data quality checklist' ],
        removed: [ ]
      },
      work_titles: { added: [ ], removed: [ ] },
      activities: { added: [ ], removed: [ ] }
    }
  }
},
```

(b) original to v3

```
{
  student_key: 'u_0001',
  comparison_type: 'step',
  compared_versions: { from: 'v1', to: 'v2' },
  focus_area: null,
  score_before: { academic: 7.64, skill: 6.15, experience: 4.025 },

  score_after: { academic: 7.715, skill: 8.35, experience: 4.025 },

  score_delta: { academic: 0.075, skill: 2.2, experience: 0 },
  improved_in_focus_area: null,
  focus_area_delta: null,
  changes: {
    academics: {
      cgpa_old: 3.46,
      cgpa_new: 3.51,
      cgpa_delta: 0.05,
      coursework: { added: [ ], removed: [ ] },
      awards: { added: [ ], removed: [ ] }
    },
    skills: {
      technical: { added: [ 'power bi', 'ci/cd' ], removed: [ ] },
      soft: { added: [ 'problem solving' ], removed: [ ] }
    },
    experience: {
      projects: { added: [ ], removed: [ ] },
      work_titles: { added: [ ], removed: [ ] },
      activities: { added: [ ], removed: [ ] }
    }
  }
},
```

(c) v1 to v2

```
{
  student_key: 'u_0001',
  comparison_type: 'step',
  compared_versions: { from: 'v2', to: 'v3' },
  focus_area: null,
  score_before: { academic: 7.715, skill: 8.35, experience: 4.025 },

  score_after: { academic: 8.29, skill: 8.35, experience: 4.825 },
  score_delta: { academic: 0.575, skill: 0, experience: 0.8 },
  improved_in_focus_area: null,
  focus_area_delta: null,
  changes: {
    academics: {
      cgpa_old: 3.51,
      cgpa_new: 3.56,
      cgpa_delta: 0.05,
      coursework: { added: [ 'applied analytics' ], removed: [ ] },
      awards: { added: [ ], removed: [ ] }
    },
    skills: {
      technical: { added: [ 'pandas' ], removed: [ ] },
      soft: { added: [ ], removed: [ ] }
    },
    experience: {
      projects: { added: [ 'data quality checklist' ], removed: [ ] }
    },
    work_titles: { added: [ ], removed: [ ] },
    activities: { added: [ ], removed: [ ] }
  }
},
```

(d) v2 to v3

Figure 18: Example Task 2 resume version comparisons for one student.

Additional Feature:

Running and Testing:


```

use("proj");

showVersionDiff("u_0001", "original", "v2");

```

Output:

```

{
  student_key: 'u_0001',
  compared_versions: {
    from: 'original',
    to: 'v2'
  },
  compared_timestamps: {
    from: ISODate('2026-02-03T10:14:22.000Z'),
    to: ISODate('2026-06-30T23:52:22.000Z')
  },
  focus_area: null,
  score_before: {
    academic: 7.065,
    skill: 5.1,
    experience: 3.225
  },
  score_after: {
    academic: 7.715,
    skill: 8.35,
    experience: 4.025
  },
  score_delta: {
    academic: 0.65,
    skill: 3.25,
    experience: 0.8
  },
  improved_in_focus_area: null,
  focus_area_delta: null,
  changes: {
    academics: {
      cgpa_old: 3.41,
      cgpa_new: 3.51,
      cgpa_delta: 0.1,
      coursework: {
        added: [
          'project management'
        ],
        removed: []
      },
      awards: {
        added: [],
        removed: []
      }
    },
  },
}

```

(a) part1

```

skills: {
  technical: {
    added: [
      'unit testing',
      'power bi',
      'ci/cd'
    ],
    removed: []
  },
  soft: {
    added: [
      'problem solving'
    ],
    removed: []
  }
},
experience: {
  projects: {
    added: [
      'resume change log script'
    ],
    removed: []
  },
  work_titles: {
    added: [],
    removed: []
  },
  activities: {
    added: [],
    removed: []
  }
},
score_delta: {
  academic: 0.65,
  skill: 3.25,
  experience: 0.8
}
}

```

(b) part2

Task 3

Goal: You have a strong resume, but every job posting is different. How do you know which of your experiences and projects are actually relevant to the role you are applying for? Task 3 can take your full resume and a target job description, then automatically surface the most relevant bullets and items so your one-page resume is always tailored to the right audience.

Main Files:

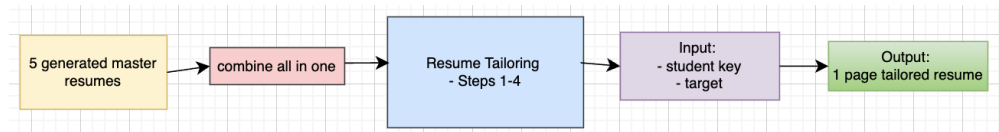
Inside the task3 folder:

- task3finalcode.mongosh.js – main aggregation pipeline, defines and runs the `runTailor()` function
- input – two example calls showing how to run the tailor for different target roles

Outside task3, in the main repo:

- masterResume – folder with 5 master resumes [containing every detail], same student keys as the 20 generated students

How it Works:



Step 1: Flatten the Resume into bullets

The pipeline starts by matching the student document from `master_resumes` using `student_key`. It then uses `$concatArrays` and `$map` to flatten all experience, project, and research entries into a single `all_items` array. Each item carries its section label, title, org, dates, and bullet list. The pipeline then `$unwinds` twice, once on `all_items` and once on `all_items.bullets`, so every individual bullet becomes its own document for scoring.

This is necessary because MongoDB cannot score individual bullets while they are still nested inside arrays.

```

$addFields: {
  all_items: {
    $concatArrays: [
      {
        $map: {
          input: { $ifNull: ["$experience", []] },
          as: "e",
          in: {
            section: "experience",
            title: "$$e.role_title",
            org: "$$e.company",
            start_date: "$$e.start_date",
            end_date: "$$e.end_date",
            bullets: { $ifNull: ["$e.bullets", []] }
          }
        }
      },
      {
        $map: {
          input: { $ifNull: ["$projects", []] },
          as: "p",
          in: {
            section: "projects",
            title: "$$p.project_name",
            org: "$$p.category",
            start_date: "$$p.start_date",
            end_date: "$$p.end_date",
            bullets: { $ifNull: ["$p.bullets", []] }
          }
        }
      },
      {
        $map: {
          input: { $ifNull: ["$research", []] },
          as: "r",
          in: {
            section: "research",
            title: "$$r.role_title",
            org: "$$r.institution",
            start_date: "$$r.start_date",
            end_date: "$$r.end_date",
            bullets: { $ifNull: ["$r.bullets", []] }
          }
        }
      }
    ]
  }
}

```

Step 2: Score each bullet

Each bullet document gets a `combined_text` field by concatenating the item title, org, and bullet text in lowercase. Two regex patterns are then matched against this text:

- **targetHits**: how many times the target job keywords appear in the bullet (e.g. "data analyst internship sql python healthcare" → keywords extracted and matched)
- **task0Hits**: how many times Task 0's domain keywords appear (internship, projects, cgpa, communication, leadership, etc.)

Bullets that have zero target keyword hits are filtered out with `$match`. The remaining bullets are scored as:

$$\text{score} = (\text{targetHits} \times 2) + \text{task0Hits}$$

Target keywords are weighted double because relevance to the specific role matters more than general placement signal. Bullets are sorted descending by score.

```

    $addFields: {
      bullet_text: "$all_items.bullets",
      targetHits: { $size: { $regexFindAll: { input: "$combined_text", regex: targetRe } } },
      task0Hits: { $size: { $regexFindAll: { input: "$combined_text", regex: task0Re } } }
    }
  },

  { $match: { targetHits: { $gt: 0 } } },

  { $addFields: { score: { $add: [{ $multiply: ["$targetHits", 2] }, "$task0Hits"] } } },
  { $sort: { score: -1 } },

```

Step 3: Regroup and Trim

After scoring, bullets are regrouped back to their parent item using **\$group** on section, title, org, and dates. Only the top **bulletsPerItem** bullets (default: 2) are kept per item using **\$slice**. Items are then sorted by their **bestScore** and limited to **maxItemsTotal** (default: 8) across all sections. This produces a ranked, trimmed list of the most relevant resume items for the target role.

```

    $group: {
      _id: {
        section: "$all_items.section",
        title: "$all_items.title",
        org: "$all_items.org",
        start_date: "$all_items.start_date",
        end_date: "$all_items.end_date"
      },
      bullets: { $push: "$bullet_text" },
      bestScore: { $max: "$score" },
      education: { $first: "$education" },
      skills: { $first: "$skills" }
    }
  },

  {
    $project: {
      _id: 0,
      section: "$_id.section",
      title: "$_id.title",
      org: "$_id.org",
      start_date: "$_id.start_date",
      end_date: "$_id.end_date",
      bullets: { $slice: ["$bullets", bulletsPerItem] },
      education: 1,
      skills: 1,
      bestScore: 1
    }
  },

  { $sort: { bestScore: -1 } },
  { $limit: maxItemsTotal },

```

Step 4: Assemble the Tailored Resume

A final **\$group** and **\$project** stage reassembles the output into a clean resume object. Education and skills are passed through from the original document. The selected items are split back into their sections (experience, projects, research) using **\$filter**. The result is a single JSON document structured as a ready-to-use one-page resume tailored to the target role.

```

    $group: {
      _id: studentKey,
      education: { $first: "$education" },
      skills: { $first: "$skills" },
      items: {
        $push: {
          section: "$section",
          title: "$title",
          org: "$org",
          start_date: "$start_date",
          end_date: "$end_date",
          bullets: "$bullets"
        }
      }
    },
  },
  {
    $project: {
      _id: 0,
      student_key: "$_id",
      target: target,
      resume: {
        education: "$education",
        skills: "$skills",
        experience: { $filter: { input: "$items", as: "it", cond: { $eq: ["$$it.section", "experience"] } } },
        projects: { $filter: { input: "$items", as: "it", cond: { $eq: ["$$it.section", "projects"] } } },
        research: { $filter: { input: "$items", as: "it", cond: { $eq: ["$$it.section", "research"] } } }
      }
    }
  }
}).toArray();

```

Running and Testing:

```
runTailor("u_0001", "data analyst internship sql python healthcare")
```

Output:

```

{
  student_key: 'u_0001',
  target: 'data analyst internship sql python healthcare',
  resume: {
    education: [
      {
        institution: 'University of California, Davis',
        degree: 'M.S.',
        field_of_study: 'Data Science',
        start_date: '2024-09',
        end_date: '2026-06',
        status: 'in_progress',
        cgpa: 3.82,
        coursework: [
          {
            code: 'DSC 241',
            name: 'Statistical Models',
            topics: [
              'multiple regression',
              'diagnostics',
              'model selection (AIC/CV)',
              'bootstrap',
              'generalized linear models',
              'penalized regression',
              'nonparametric regression'
            ]
          },
          {
            code: 'ECS 171',
            name: 'Machine Learning'
          },
          {
            code: 'STA 207',
            name: 'Applied Regression'
          },
          {
            code: 'ECS 272',
            name: 'Information Visualization'
          }
        ],
        highlights: [
          'Teaching support role (2025): led lab sections/office hours and helped grade assignments.'
        ]
      }
    ]
  }
}

```

Figure 20: Tailored resume output, part 1

```

skills: {
  programming_languages: [
    'Python',
    'SQL',
    'R',
    'JavaScript',
    'TypeScript'
  ],
  ml_data: [
    'pandas',
    'numpy',
    'scikit-learn',
    'statsmodels',
    'xgboost (basic)',
    'PyTorch (basic)'
  ],
  visualization: [
    'matplotlib',
    'plotly',
    'Tableau',
    'Power BI'
  ],
  data_engineering: [
    'ETL (Python/SQL)',
    'Airflow (basic)',
    'dbt (basic)'
  ],
  databases: [
    'PostgreSQL',
    'MySQL',
    'SQLite',
    'BigQuery (basic)'
  ],
  cloud_tools: [
    'AWS (S3, Lambda) (basic)',
    'Docker (basic)',
    'GitHub Actions'
  ],
  soft_skills: [
    'technical writing',
    'presentation',
    'stakeholder communication',
    'team collaboration'
  ]
},

```

(a) Tailored resume output, part 2

```

experience: [
  {
    section: 'experience',
    title: 'Data Science Intern',
    org: 'Nimbus Health Analytics',
    start_date: '2025-06',
    end_date: '2025-09',
    bullets: [
      'Designed a feature pipeline in SQL + Python (120+ features) with automated data quality checks and reproducible notebooks.',
      'Built a claims-based patient risk model using logistic regression and gradient boosting; improved AUC from 0.71 to 0.79 on validation data.'
    ]
  },
  {
    section: 'experience',
    title: 'Student Developer',
    org: 'Campus IT Services (UCSD)',
    start_date: '2022-01',
    end_date: '2023-06',
    bullets: [
      'Built an internal ticket triage tool integrating email + database records; reduced manual tagging workload by ~25%.',
      'Wrote SQL queries and exports for monthly service metrics used in reporting.'
    ]
  }
],
projects: [
  {
    section: 'projects',
    title: 'Personal Finance Dashboard',
    org: 'personal_project',
    start_date: '2023-08',
    end_date: '2023-12',
    bullets: [
      'Ingested transaction data and built automated category rules; created interactive spending trend dashboards.'
    ]
  }
],
research: []
}

```

(b) Tailored resume output, part 3

Figure 21: Example Task 3 tailored resume output for one student and target role.

Same student key [0001] but different target:

Input:

```
runTailor("u_0001", " IT services")
```

Output:

```

{
  student_key: 'u_0001',
  target: 'IT services',
  resume: {
    education: [
      {
        institution: 'University of California, Davis',
        degree: 'M.S.',
        field_of_study: 'Data Science',
        start_date: '2024-09',
        end_date: '2026-06',
        status: 'in_progress',
        cgpa: 3.82,
        coursework: [
          {
            code: 'DSC 241',
            name: 'Statistical Models',
            topics: [
              'multiple regression',
              'diagnostics',
              'model selection (AIC/CV)',
              'bootstrap',
              'generalized linear models',
              'penalized regression',
              'nonparametric regression'
            ]
          },
          {
            code: 'ECS 171',
            name: 'Machine Learning'
          },
          {
            code: 'STA 207',
            name: 'Applied Regression'
          },
          {
            code: 'ECS 272',
            name: 'Information Visualization'
          }
        ],
        highlights: [
          'Teaching support role (2025): led lab sections/office hours and helped grade assignments.'
        ]
      }
    ],
  },
}

```

```

{
  skills: {
    programming_languages: [
      'Python',
      'SQL',
      'R',
      'JavaScript',
      'TypeScript'
    ],
    ml_data: [
      'pandas',
      'numpy',
      'scikit-learn',
      'statsmodels',
      'xgboost (basic)',
      'PyTorch (basic)'
    ],
    visualization: [
      'matplotlib',
      'plotly',
      'Tableau',
      'Power BI'
    ],
    data_engineering: [
      'ETL (Python/SQL)',
      'Airflow (basic)',
      'dbt (basic)'
    ],
    databases: [
      'PostgreSQL',
      'MySQL',
      'SQLite',
      'BigQuery (basic)'
    ],
    cloud_tools: [
      'AWS (S3, Lambda) (basic)',
      'Docker (basic)',
      'GitHub Actions'
    ],
    soft_skills: [
      'technical writing',
      'presentation',
      'stakeholder communication',
      'team collaboration'
    ]
  },
  experience: [
    {
      section: 'experience',
      title: 'Student Developer',
      org: 'Campus IT Services (UCSD)',
      start_date: '2022-01',
      end_date: '2023-06',
      bullets: [
        'Built an internal ticket triage tool integrating email + database records; reduced manual tagging workload by ~25%.',
        'Wrote SQL queries and exports for monthly service metrics used in reporting.'
      ]
    }
  ],
  projects: [],
  research: []
}

```

Task 4

Goal: Task 4 evaluates a student's resume and determine how competitive it is compared to others. The system first converts resumes into structured scores for academics, skills, and experience (from Task 1). These scores are then combined with lift values from the larger Task 0 dataset to compute a placement prediction that estimates the student's placement potential.

Because resumes contain unstructured text, traditional relational queries cannot effectively measure similarity between profiles. By using a vector database to perform nearest-neighbor search, the system identifies the most similar resumes among the 20 students to show where the student stands within their peer group rather than against the entire dataset. It provides more relevant comparisons and insights into which attributes may improve placement outcomes.

It then compares the student against stronger candidates from the larger dataset to highlight performance gaps using PostgreSQL. This result shows which areas of (academics, skills, or experience) should be improved to increase placement competitiveness.

Source File: 202-Final-Project/task4 /Task4_Final.ipynb

How it Works: finding top 5 similar students using ChromaDB

Step 1: Import libraries and connect to ChromaDB and load the Task 1 20 resumes json files.

```
# Load scored profiles for resumes from Task 1
# !!!!!!! PASTE OWN FILE PATH FOR 'task1_final_output.json'!!!!!!!
with open('', 'r') as f:
    task1 = json.load(f)
print(f'Task1 profiles loaded: {len(task1)}')
```


Step 2: Convert Task 1 20 resumes into text string

```
# # Convert one resume dict into a single text string for embedding
def resume_to_text(resume: dict) -> str:
    major = resume.get('college', {}).get('major', '')
    cgpa = resume.get('academics', {}).get('cgpa', '')
    courses = ', '.join([
        c.get('course', '')
        for c in resume.get('academics', {}).get('relevant_coursework', [])
    ])
    awards = ', '.join([
        a.get('title', '')
        for a in resume.get('academics', {}).get('honors_or_awards', [])
    ])
    tech = ', '.join(resume.get('skills', {}).get('technical', []))
    soft = ', '.join(resume.get('skills', {}).get('soft', []))
    projects = ', '.join([
        p.get('name', '')
        for p in resume.get('experience', {}).get('projects', [])
    ])
    work = ', '.join([
        w.get('title', '')
        for w in resume.get('experience', {}).get('work', [])
    ])
    activities = ', '.join([
        a.get('activity', '')
        for a in resume.get('experience', {}).get('leadership_and_activities', [])
    ])
    return (
        f'Major: {major}. CGPA: {cgpa}. '
        f'Courses: {courses}. Awards: {awards}. '
        f'Technical skills: {tech}. Soft skills: {soft}. '
        f'Projects: {projects}. Work: {work}. Activities: {activities}.'
    )
```

Step 3: Vector Embedding and store 20 resumes

```
# Build rich-text vector collection from 20 resumes
print('Building collection_rich (20 resume-text vectors)...')
texts_rich = [resume_to_text(r) for r in resumes]
ids_rich = [r['student_key'] for r in resumes]
embeddings_rich = model.encode(texts_rich).tolist()
metadatas_rich = [
    {
        'student_key': r['student_key'],
        'major': r['college']['major'],
        'cgpa': str(r['academics']['cgpa']),
        'source': 'task1_resume'
    }
    for r in resumes
]

# Store rich resume embeddings in ChromaDB
collection_rich.add(
    ids=ids_rich,
    embeddings=embeddings_rich,
    metadatas=metadatas_rich,
    documents=texts_rich
)
```

Step 4: Convert one of the uploaded resume into a text representation and generate its embedding vector.

```
query_resume_7 = next(r for r in resumes if r['student_key'] == 'u_0007')
query_profile_7 = next(p for p in task1_profiles if p['student_key'] == 'u_0007')
run_task3(query_resume_7, query_profile_7)
```

Step 5: Query ChromaDB for top 5 similar students and returns the nearest neighbors by cosine distance. Lower distance means more similar.

```
# Find the most similar resumes from the 20 rich-text profiles
def find_rich_similar(query_resume: dict, k: int = 5) -> list:
    query_text = resume_to_text(query_resume)
    query_emb = model.encode([query_text]).tolist()
    student_key = query_resume['student_key']

    # Query ChromaDB and exclude the same student
    results = collection_rich.query(
        query_embeddings=query_emb,
        n_results=k + 1,  # fetch one extra to account for self
        where={'student_key': {'$ne': student_key}}, # exclude self
        include=['distances', 'metadatas']
    )

    # Return top k matches with key metadata
    return [
        {
            'student_key': results['ids'][0][i],
            'distance': round(results['distances'][0][i], 4),
            'major': results['metadatas'][0][i]['major'],
            'cgpa': results['metadatas'][0][i]['cgpa'],
            'collection': 'rich (resume text)',
        }
        for i in range(min(k, len(results['ids'][0]))) # cap at k
    ]
```

****To compute placement prediction using PostgreSQL**

Step 1:

Load Task 0 JSON data. Reusing the 20 resumes from the Task 1 JSON dataset, each student already has academic, skill, and experience scores. A mapping dictionary is used to map to the corresponding Task 0 major names because the naming conventions between Task 1 and Task 0 may differ. Thus the appropriate lift values from Task 0 can be retrieved for each major later.

```
# Load Task 0 output
# !!!!!!! PASTE OWN FILE PATH FOR 'postgres_public_task0_final_output.json'!!!!!!!
with open('', 'r') as f:
    task0_all = json.load(f)
print(f'Task0 records loaded: {len(task0_all)}')

# Keep only Task 0 output
major_lifts = {}
for r in task0_all:
    if r['major'] and r['major'] not in major_lifts:
        major_lifts[r['major']] = {
            'academic_lift': r['academic_lift'] if r['academic_lift'] is not None else 0,
            'skill_lift': r['skill_lift'] if r['skill_lift'] is not None else 0,
            'experience_lift': r['experience_lift'] if r['experience_lift'] is not None else 0
        }

# Map Task 1 major names to Task 0 major names
task0 = [
    r for r in task0_all
    if r['major']
    and r['academic_bucket']
    and r['skill_bucket']
    and r['experience_bucket']
    and r['academic_score'] is not None
    and r['skill_score'] is not None
    and r['experience_score'] is not None
]
print(f'Task0 usable records: {len(task0)}')
```

Step 2:

After this mapping step, the Task 1 student scores are combined with the Task 0 lift values to compute the placement prediction later.

```
# Build lookup of lift values by Task 0 major
task1_profiles = []
for s in task1:
    task0_major = MAJOR_MAP.get(s['major'], s['major'])
    lifts = major_lifts.get(task0_major, {
        'academic_lift': 0, 'skill_lift': 0, 'experience_lift': 0
    })
    # Combine Task 1 scores with mapped Task 0 lift values
    task1_profiles.append({
        'student_key': s['student_key'],
        'cgpa': s['cgpa'],
        'major': s['major'],
        'task0_major': task0_major,
        'academic_score': s['academic_score'],
        'skill_score': s['skill_score'],
        'experience_score': s['experience_score'],
        'academic_lift': lifts['academic_lift'],
        'skill_lift': lifts['skill_lift'],
        'experience_lift': lifts['experience_lift'],
    })

# Collect all scores from Task 0 and Task 1 for shared normalization
all_acad = [r['academic_score'] for r in task0] + [p['academic_score'] for p in task1_profiles]
all_skill = [r['skill_score'] for r in task0] + [p['skill_score'] for p in task1_profiles]
all_exp = [r['experience_score'] for r in task0] + [p['experience_score'] for p in task1_profiles]

# Store min/max ranges for later normalization
NORM = {
    'acad_min': min(all_acad), 'acad_max': max(all_acad),
    'skill_min': min(all_skill), 'skill_max': max(all_skill),
    'exp_min': min(all_exp), 'exp_max': max(all_exp),
}
```

Step 3: Compute placement prediction

raw score = (academic score × academic lift) + (skill score × skill lift) + (experience score × experience lift)

```
# Compute placement prediction from normalized scores and lift weights
# Returns placement_prediction as a percentage (0-100).
def compute_placement_prediction(academic_score, skill_score, experience_score,
                                academic_lift, skill_lift, experience_lift) -> float:

    # Normalize scores to 0-1
    acad_n = normalize_score(academic_score, NORM['acad_min'], NORM['acad_max'])
    skill_n = normalize_score(skill_score, NORM['skill_min'], NORM['skill_max'])
    exp_n = normalize_score(experience_score, NORM['exp_min'], NORM['exp_max'])

    # Clamp negative lifts to 0
    al = max(academic_lift, 0)
    sl = max(skill_lift, 0)
    el = max(experience_lift, 0)
    total_lift = al + sl + el

    # Use equal-weight average if all lifts are 0
    if total_lift == 0:
        return round(((acad_n + skill_n + exp_n) / 3.0) * 100, 1)

    # Compute lift-weighted percentage
    raw = (acad_n * al) + (skill_n * sl) + (exp_n * el)
    return round((raw / total_lift) * 100, 1)
```

Step 4: Normalize the data

The scores are normalized to a 0–1 scale so that academic, skill, and experience scores are comparable and no category dominates the placement prediction due to having larger numeric ranges. Normalized the score calculation is $(\text{value} - \text{min}) / (\text{max} - \text{min})$.

```
# # Scale a score to the 0–1 range
def normalize_score(value: float, min_val: float, max_val: float) -> float:
    if max_val == min_val:
        return 0.0
    return round((value - min_val) / (max_val - min_val), 6)

# Convert structured profile scores into text for embedding
def profile_to_text(major: str, academic_score: float,
                   skill_score: float, experience_score: float,
                   academic_bucket: str, skill_bucket: str,
                   experience_bucket: str) -> str:
    acad_n = normalize_score(academic_score, NORM['acad_min'], NORM['acad_max'])
    skill_n = normalize_score(skill_score, NORM['skill_min'], NORM['skill_max'])
    exp_n = normalize_score(experience_score, NORM['exp_min'], NORM['exp_max'])
    return (
        f'Major: {major}. '
        f'Academic score: {acad_n:.3f} ({academic_bucket}). '
        f'Skill score: {skill_n:.3f} ({skill_bucket}). '
        f'Experience score: {exp_n:.3f} ({experience_bucket}). '
    )
```

Result:

Data from Task 0 and Task 1:

```
Resumes loaded: 20
Task1 profiles loaded: 20
Task0 records loaded: 2000
Task0 usable records: 2000

Normalization ranges (combined task0 + task1):
academic:  5.873 – 10.000
skill:     5.100 – 30.929
experience: 0.000 – 6.667
```

The system loads 20 resume profiles from Task 1 and 2,000 usable records from the Task 0 dataset. These records are used to compute normalization ranges by combining score values from both datasets. The ranges show the minimum and maximum values observed for each academic, skill, and experience scores. These ranges are later used to normalize the scores to a common scale before computing the placement prediction.

Student u_0007 Result:

Student Key – u_0007 | Computer Science | CGPA: 3.62
Scores: acad=7.38 skill=5.1 exp=3.23
Buckets: High / Low / Low
placement_prediction: 28.9%

OUTPUT 1 – Top 5 Similar Resume Peers (collection_rich)

Question: "Which of the 20 detailed resume peers is most similar to me?"

Method: Full resume text embedding (semantic similarity)

student_key	distance	major	cgpa	academic_score	skill_score	experience_score	placement_prediction
u_0020	0.0469	Computer Science	3.95	7.8800	6.1500	3.2300	30.6000%
u_0004	0.0542	Data Science	3.88	7.7700	5.1000	3.0500	35.3000%
u_0016	0.0628	Data Science	3.33	6.9500	5.1000	3.2300	30.8000%
u_0001	0.0711	Computer Science	3.41	7.0700	5.1000	3.2300	28.9000%
u_0013	0.0786	Computer Science	3.47	7.1600	5.1000	3.2300	28.9000%

OUTPUT 2 – Top 3 Stronger Comparison Students (PostgreSQL)

Question: "Who has better placement odds than me in CSE?"

NOTE: Queried across ALL 2,020 students – not filtered from Output 1.

Your placement percentile: top 93.7% (outperform 127 of 2020 students)

student_key	major	academic_score	skill_score	experience_score	placement_prediction
d3_469	CSE	8.3600	25.2503	5.3333	79.2%
d3_295	CSE	8.6850	25.7148	4.6667	74.0%
d3_1493	CSE	7.8853	23.8216	5.0000	74.0%

IMPROVEMENT DIRECTION (vs top stronger peer):

- ▲ Academic Score : you=36.5% | peer=60.3% | gap=+23.7%
- ▲ Skill Score : you=0.0% | peer=78.0% | gap=+78.0%
- ▲ Experience Score: you=48.4% | peer=80.0% | gap=+31.5%

The system evaluates student u_0007, a Computer Science major with a CGPA of 3.62. Based on the resume data processed in Task 1, the student receives an academic score of 7.38, a skill score of 5.10, and an experience score of 3.23. These scores are normalized and combined with major-specific lift values derived from the Task 0 dataset to compute a **placement prediction of 28.9%**, indicating that the student's overall profile is relatively weak compared to the highest performing placement profiles in the dataset.

Using the **vector database**, the system identifies the **five most similar resumes** among the **20 resumes from Task 1**. These peers have comparable resume characteristics and **placement predictions ranging from 29% to 35%**, suggesting that the student belongs to a peer group with similar academic and resume profiles. However, when compared with the broader Task 0 dataset of approximately 2,020 students, several stronger peers have significantly higher placement predictions, with the strongest reaching **79.2%**.

Further comparison of the normalized scores shows that the largest performance gaps occur in **skills and experience**. The **skill score gap** between the student and the strongest peer is approximately **78%**, while the **experience score gap** is about **31.5%**. This indicates that although the student has a reasonable academic profile, improving **technical skills** and gaining more **project or internship experience** would most effectively increase the student's placement competitiveness.

Student u_0020 Result:

Student Key – u_0020 | Computer Science | CGPA: 3.95
Scores: acad=7.88 skill=6.15 exp=3.23
Buckets: High / High / High
placement_prediction: 30.6%

OUTPUT 1 – Top 5 Similar Resume Peers (collection_rich)

Question: "Which of the 20 detailed resume peers is most similar to me?"

Method: Full resume text embedding (semantic similarity)

student_key	distance	major	cgpa	academic_score	skill_score	experience_score	placement_prediction
u_0007	0.0469	Computer Science	3.62	7.3800	5.1000	3.2300	28.9000%
u_0013	0.0713	Computer Science	3.47	7.1600	5.1000	3.2300	28.9000%
u_0001	0.0716	Computer Science	3.41	7.0700	5.1000	3.2300	28.9000%
u_0017	0.0833	Computer Science	2.94	6.3600	5.1000	3.2300	28.9000%
u_0004	0.0847	Data Science	3.88	7.7700	5.1000	3.0500	35.3000%

OUTPUT 2 – Top 3 Stronger Comparison Students (PostgreSQL)

Question: "Who has better placement odds than me in CSE?"

NOTE: Queried across ALL 2,020 students – not filtered from Output 1.

Your placement percentile: top 90.4% (outperform 193 of 2020 students)

student_key	major	academic_score	skill_score	experience_score	placement_prediction
d3_469	CSE	8.3600	25.2503	5.3333	79.2%
d3_295	CSE	8.6850	25.7148	4.6667	74.0%
d3_1493	CSE	7.8853	23.8216	5.0000	74.0%

IMPROVEMENT DIRECTION (vs top stronger peer):

- ▲ Academic Score : you=48.6% | peer=60.3% | gap=+11.6%
- ▲ Skill Score : you=4.1% | peer=78.0% | gap=+73.9%
- ▲ Experience Score: you=48.4% | peer=80.0% | gap=+31.5%

The system evaluates student u_0020, a Computer Science major with a CGPA of 3.95. Based on the resume data from Task 1, the student receives an academic score of 7.88, skill score of 6.15, and experience score of 3.23, which places the student in the High bucket for all three categories. After normalizing the scores and applying the major-specific lift weights learned from the Task 0 dataset, the model computes a **placement prediction of 30.6%**.

The system identifies the five most similar resume profiles and similar academic and experience scores and placement predictions around 28–35%. Stronger peers have significantly **higher placement predictions of 79.2%**. The improvement analysis shows that the largest gap occurs in the skill score which is **73.9%**, followed by experience (**31.5%**), while the academic score gap is relatively smaller (**11.6%**). This suggests that the student already has a strong academic background, hence **improving technical and experience skills** would be the most effectively increase the student's placement competitiveness.

Task 0, 1, 4 & 5 Connection:

Task 4 connects the results of Task 0 and Task 1 by combining the lift values computed from the Task 0 dataset with the structured resume scores computed in Task 1. Using these values, the system calculates a placement prediction score that estimates how competitive a resume is relative to the dataset.

A vector database is used to find similar resume profiles among the 20 students, while PostgreSQL identifies stronger peers from the broader dataset.

While Task 4 compares resumes using similarity search and placement predictions, it treats students as independent records. These results can be expanded in Task 5 to capture relationships between students, attributes, and outcomes, allowing the system to traverse peer connections and generate recommendations based on shared patterns among placed students.

Task 5

Goal:

Task 5 extends the analysis by modeling students, attributes, and placement outcomes as a graph in Neo4j. Instead of treating each resume as an independent record. The graph structure connects students to shared attribute buckets through academic level, skill level, and experience level.

Lift values computed from Task 0 are stored as properties and used during the analysis to evaluate which attributes most influence placement outcomes. The system creates similarity relationships between students and enables multi-hop traversal through the peer network.

Main Files: 202-Final-Project/task5 /task_5_final_graph.ipynb

Due to computational and resource limits of the free Neo4j Aura version, the similarity edge computation was limited during execution, and the number of edges written to the graph was capped. Therefore, the report focuses on the CSE major as a representative example for demonstrating the graph analysis.

Steps for constructing the graph, result and visualization:

Step 1: We first filtered the Task 0 output from 12,922 records to 2,000 usable students by removing rows with null major or missing bucket values. For each remaining student, we ran one Cypher MERGE query to create a Student node and connect it to one shared Major node, one AcademicBucket node, one SkillBucket node, one ExperienceBucket node, and one Outcome node. This gives 6 nodes involved and 5 relationships per student, non-student nodes may be reused across students through MERGE.

Here are the examples of using 3 students in the same CS major, D3 100, D3 1002 and D3 1005.

```
{
  "student_key": "d3_100",
  "major": "CSE",
  "academic_score": 6.8344243163550835,
  "skill_score": 16.2139820699943811,
  "experience_score": 1.3333333333333333,
  "academic_bucket": "Medium",
  "skill_bucket": "High",
  "experience_bucket": "Medium",
  "placed": 0,
  "placement_rate_high_academic": 0.54782608695652173913,
  "placement_rate_low_academic": 0.60606060606060606061,
  "academic_lift": -0.05823451910408432148,
  "placement_rate_high_skill": 0.63913043478260869565,
  "placement_rate_low_skill": 0.56709956709956709957,
  "skill_lift": 0.07203086768304159608,
  "placement_rate_high_experience": 0.63043478260869565217,
  "placement_rate_low_experience": 0.53246753246753246753,
  "experience_lift": 0.09796725014116318464,
  "most_important_category": "Experience"
},
{
  "student_key": "d3_1002",
  "major": "CSE",
  "academic_score": 7.0910250225871225,
  "skill_score": 18.9986033699044158,
  "experience_score": 2.6666666666666667,
  "academic_bucket": "Medium",
  "skill_bucket": "High",
  "experience_bucket": "High",
  "placed": 0,
  "placement_rate_high_academic": 0.54782608695652173913,
  "placement_rate_low_academic": 0.60606060606060606061,
  "academic_lift": -0.05823451910408432148,
  "placement_rate_high_skill": 0.63913043478260869565,
  "placement_rate_low_skill": 0.56709956709956709957,
  "skill_lift": 0.07203086768304159608,
  "placement_rate_high_experience": 0.63043478260869565217,
  "placement_rate_low_experience": 0.53246753246753246753,
  "experience_lift": 0.09796725014116318464,
  "most_important_category": "Experience"
},
```

```

{
  "student_key": "d3_1005",
  "major": "CSE",
  "academic_score": 8.1825889201508805,
  "skill_score": 23.8122314037060058,
  "experience_score": 3.666666666666667,
  "academic_bucket": "High",
  "skill_bucket": "High",
  "experience_bucket": "High",
  "placed": 1,
  "placement_rate_high_academic": 0.54782608695652173913,
  "placement_rate_low_academic": 0.60606060606060606061,
  "academic_lift": -0.05823451910408432148,
  "placement_rate_high_skill": 0.63913043478260869565,
  "placement_rate_low_skill": 0.56709956709956709957,
  "skill_lift": 0.07203086768304159608,
  "placement_rate_high_experience": 0.63043478260869565217,
  "placement_rate_low_experience": 0.53246753246753246753,
  "experience_lift": 0.09796725014116318464,
  "most_important_category": "Experience"
},

```

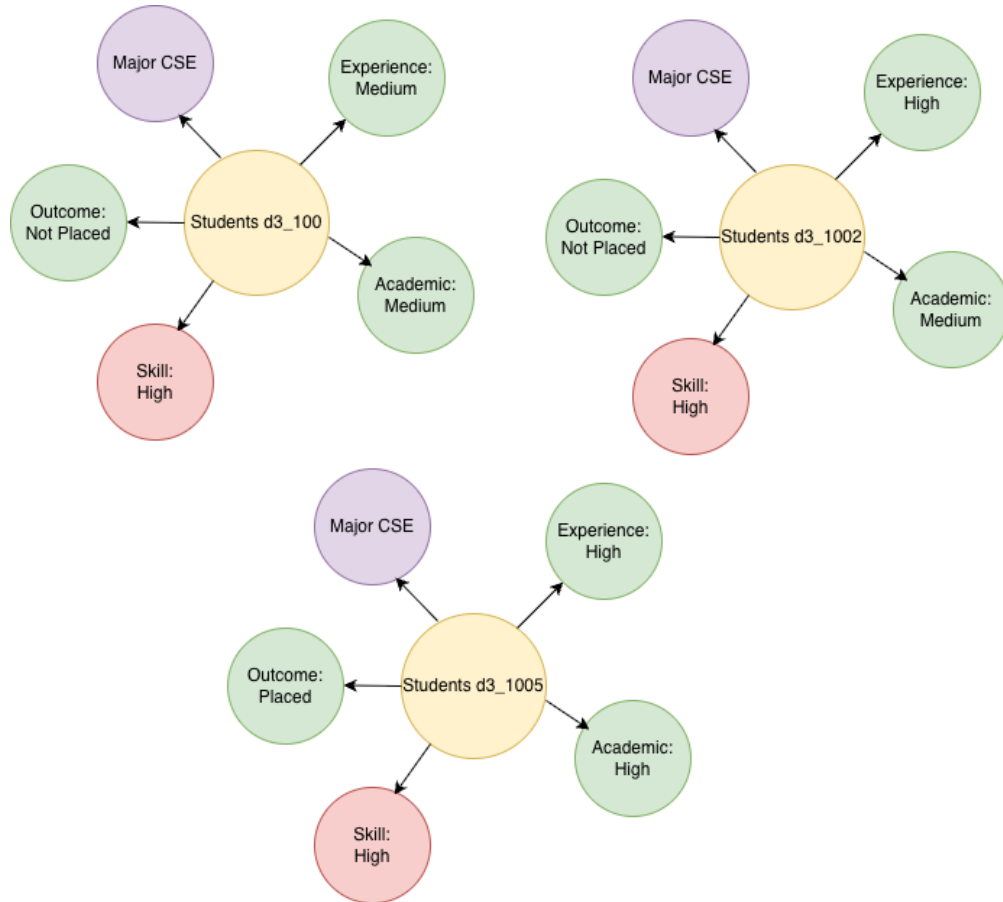


Figure 22: Base graph structure connecting each student to major, academic, skill, experience, and outcome nodes.

Step 2: Connect all nodes

```
with driver.session() as session:
    for major, lifts in major_lifts.items():
        session.run("""
            MATCH (m:Major {name: $major})
            SET m.academic_lift      = $academic_lift,
                m.skill_lift        = $skill_lift,
                m.experience_lift    = $experience_lift,
                m.most_important_category = $most_important_category
            """, major=major, **lifts)

    print(f"Lift properties set for {len(major_lifts)} majors.")

with driver.session() as session:
    nodes = session.run("MATCH (n) RETURN count(n) AS c").single()['c']
    edges = session.run("MATCH ()-[r]->() RETURN count(r) AS c").single()['c']
    print(f"Graph built: {nodes} nodes | {edges} edges | {len(students)} students")
```

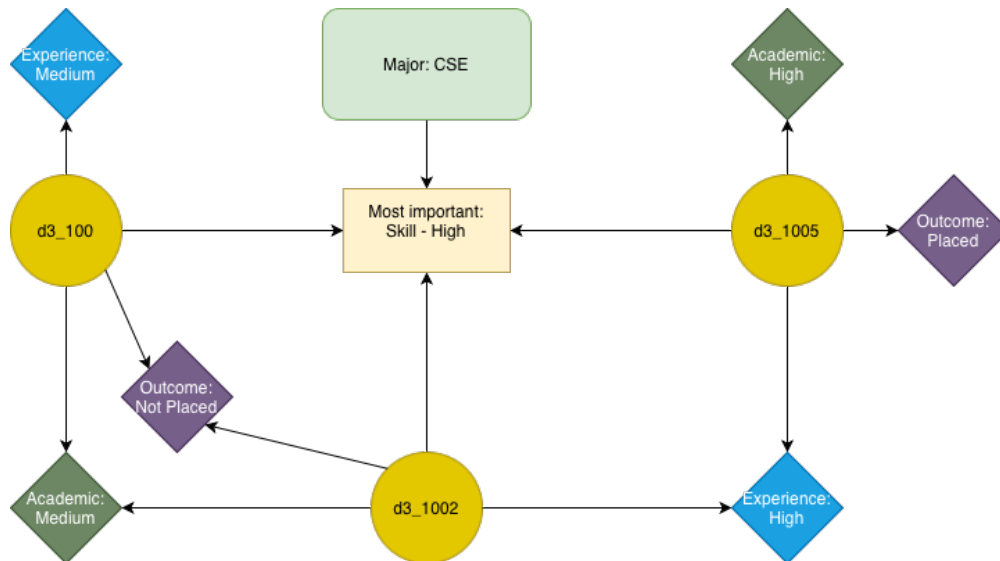


Figure 23: Connect all nodes

Step 3: Then the placement lift values for academic, skill, and experience are added as properties to each major node. These values are later used during graph traversal to analyze which attributes most influence placement outcomes. After the graph structure is built, students are compared based on their attribute buckets. If two students share at least two of the three bucket values (academic, skill, or experience), a similarity relationship is created between them, which is visualized as **red** edges in the graph.

```
#limit under Aura's 400,000 limit
all_pairs = all_pairs[:190000]
print(f"Writing {len(all_pairs)} pairs (capped to stay under Aura limit)...")

# Write in batches of 200
BATCH_SIZE = 200
for i in range(0, len(all_pairs), BATCH_SIZE):
    batch = all_pairs[i:i + BATCH_SIZE]
    driver.execute_query("""
        UNWIND $pairs AS pair
        MATCH (a:Student {student_key: pair.key_a})
        MATCH (b:Student {student_key: pair.key_b})
        MERGE (a)-[:SIMILAR_TO {shared_buckets: pair.shared}]->(b)
        MERGE (b)-[:SIMILAR_TO {shared_buckets: pair.shared}]->(a)
    """, pairs=batch)
    if (i // BATCH_SIZE + 1) % 50 == 0:
        done = min(i + BATCH_SIZE, len(all_pairs))
        print(f"  Written {done}/{len(all_pairs)} pairs...")

# Final count
total = driver.execute_query(
    "MATCH ()-[:SIMILAR_TO]->() RETURN count(r) AS c"
).records[0]['c']
print(f"\n SIMILAR_TO edges created: {total}")
```

Figure 24: Connect all nodes

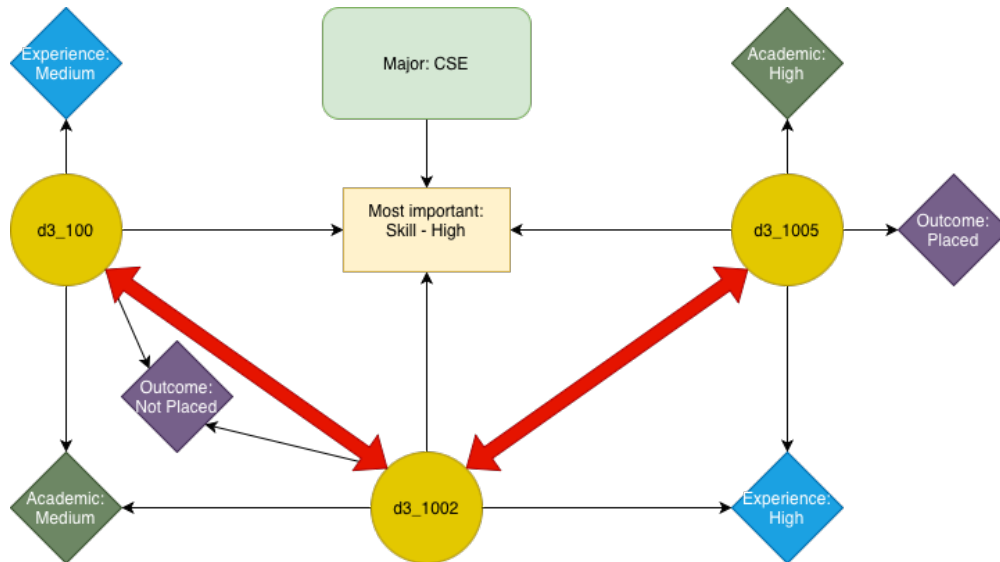


Figure 25: Create similar edges between students

Results:

```
all_pairs = []
for major, group in major_groups.items():
    print(f"Computing pairs for {major} ({len(group)} students)...")
    for i in range(len(group)):
        for j in range(i+1, len(group)):
            a, b = group[i], group[j]
            shared = sum([
                a['academic_bucket'] == b['academic_bucket'],
                a['skill_bucket'] == b['skill_bucket'],
                a['experience_bucket'] == b['experience_bucket'],
            ])
            if shared >= 2:
                all_pairs.append({
                    'key_a': a['student_key'],
                    'key_b': b['student_key'],
                    'shared': shared
                })

print(f"\nTotal pairs found: {len(all_pairs)}")
```

```
Computing pairs for CSE (691 students)...
Computing pairs for Civil (219 students)...
Computing pairs for ECE (295 students)...
Computing pairs for EEE (201 students)...
Computing pairs for IT (381 students)...
Computing pairs for Mechanical (213 students)...
```

```
Total pairs found: 296981
```

Figure 26: Number of students in each major

Centrality Analysis:

After constructing the graph, centrality analysis can be performed on the bucket nodes. We count how many placed students in a given major are connected to each bucket node (Academic, Skill, and Experience levels) and visualize the results using a bar chart.

For the CSE major, the results show that *Skill: High* is the most common attribute among placed students, followed by *Academic: High* and *Experience: High*. There are 404 placed students for *Skill: High* indicates that 404 placed CSE students have a high skill bucket.

Note that these counts are not mutually exclusive. A single student can contribute to multiple bucket counts (e.g., a student with Academic: High, Skill: High, and Experience: Medium will be counted in both Skill: High and Academic: High categories).

```
def centrality_analysis(major='CSE'):
    rows = []
    with driver.session() as session:
        for bucket_type, node_label, rel in [
            ('Academic', 'AcademicBucket', 'HAS_ACADEMIC_BUCKET'),
            ('Skill', 'SkillBucket', 'HAS_SKILL_BUCKET'),
            ('Experience', 'ExperienceBucket', 'HAS_EXPERIENCE_BUCKET'),
        ]:
            result = session.run(f"""
                MATCH (s:Student)-[:BELONGS_TO]->(m:Major {{name: $major}}),
                    (s)-[:{rel}]->(b:{node_label}),
                    (s)-[:HAS_OUTCOME]->(o:Outcome {{status: 'Placed'}})
                RETURN b.level AS level, COUNT(s) AS placed_count
                ORDER BY placed_count DESC
            """, major=major)
            for rec in result:
                rows.append({
                    'Category': bucket_type,
                    'Bucket Level': rec['level'],
                    'Placed Students': rec['placed_count'],
                    'Node Label': f"{bucket_type}:{rec['level']}"
                })

    df = pd.DataFrame(rows).sort_values('Placed Students', ascending=False)
    print(f"\nCentrality Analysis - {major}")
    print(df.to_string(index=False))
```

Centrality Analysis – CSE

Category	Bucket Level	Placed Students	Node Label
Skill	High	404	Skill:High
Academic	High	242	Academic:High
Experience	High	214	Experience:High
Experience	Medium	147	Experience:Medium
Academic	Medium	147	Academic:Medium
Experience	Low	43	Experience:Low
Academic	Low	15	Academic:Low

Figure 27: Central Analysis for CSE students

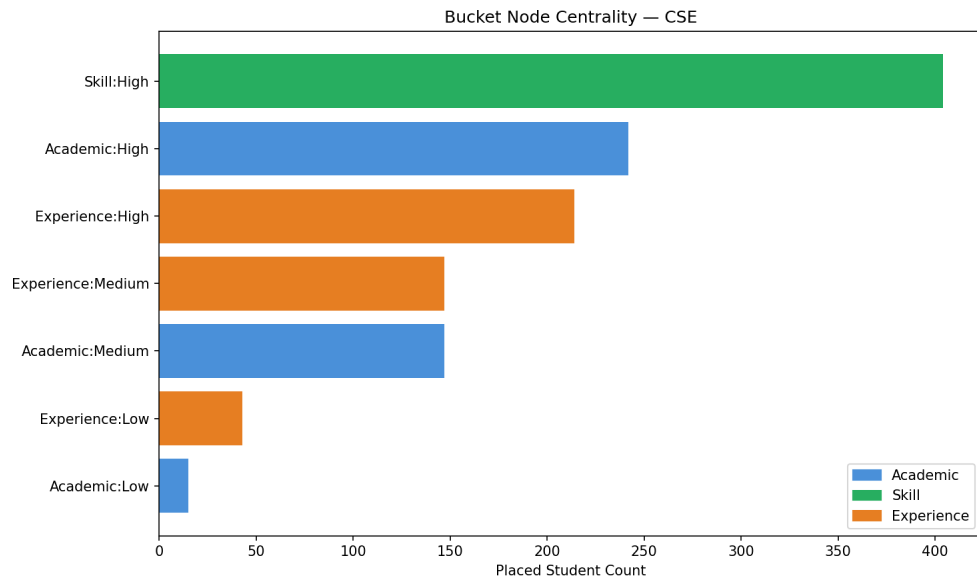


Figure 28: Bar chart for central analysis

Furthermore, we compute a multi-hop placement pattern analysis by identifying the most common bucket combinations among placed students within each major. These combinations represent the joint distribution of Academic, Skill, and Experience buckets, such as High/High/High, High/High/Medium, and Medium/High/High etc.

For each major, we group placed students by their bucket combinations and compute the number of students and the percentage of placed students belonging to each combination. The results highlight the most frequent successful profiles among placed students.

```
def run_multihop_query(major='CSE'):
    with driver.session() as session:
        total_placed = session.run("""
            MATCH (s:Student)-[:BELONGS_TO]->(:Major {name: $major}),
                (s)-[:HAS_OUTCOME]->(:Outcome {status: 'Placed'})
            RETURN COUNT(s) AS total
            """, major=major).single()['total']

        if total_placed == 0:
            print(f"\n{major}: No placed students - skipping.")
            return

        result = session.run("""
            MATCH (s:Student)-[:BELONGS_TO]->(:Major {name: $major}),
                (s)-[:HAS_ACADEMIC_BUCKET]->(ab:AcademicBucket),
                (s)-[:HAS_SKILL_BUCKET]->(sb:SkillBucket),
                (s)-[:HAS_EXPERIENCE_BUCKET]->(eb:ExperienceBucket),
                (s)-[:HAS_OUTCOME]->(:Outcome {status: 'Placed'})
            RETURN ab.level AS academic,
                sb.level AS skill,
                eb.level AS experience,
                COUNT(s) AS placed_count,
                ROUND(100.0 * COUNT(s) / $total_placed, 1) AS pct
            ORDER BY placed_count DESC
            LIMIT 10
            """, major=major, total_placed=total_placed)

        rows = [dict(r) for r in result]
        df = pd.DataFrame(rows)
        print(f"\n=== TOP PLACEMENT PATHS: {major} (total placed: {total_placed}) ===")
        print(df.to_string(index=False))

    for major in ['CSE', 'IT', 'ECE', 'Civil', 'Mechanical', 'EEE']:
        run_multihop_query(major)
```



```

=== TOP PLACEMENT PATHS: CSE (total placed: 404) ===
academic skill experience placed_count pct
  High   High      High      129 31.9
  High   High      Medium     86 21.3
  Medium High      High      80 19.8
  Medium High      Medium     54 13.4
  High   High      Low       27  6.7
  Medium High      Low       13  3.2
  Low    High      Medium      7  1.7
  Low    High      High        5  1.2
  Low    High      Low         3  0.7

=== TOP PLACEMENT PATHS: IT (total placed: 196) ===
academic skill experience placed_count pct
  High   High      High      71 36.2
  High   High      Medium     49 25.0
  Medium High      High      33 16.8
  Medium High      Medium     19  9.7
  High   High      Low       14  7.1
  Medium High      Low        6  3.1
  Low    High      High        2  1.0
  Low    High      Low         1  0.5
  Low    High      Medium      1  0.5

=== TOP PLACEMENT PATHS: ECE (total placed: 147) ===
academic skill experience placed_count pct
  High   High      High      42 28.6
  Medium High      High      32 21.8
  High   High      Medium     31 21.1
  Medium High      Medium     29 19.7
  High   High      Low        7  4.8
  Low    High      High        2  1.4
  Low    High      Medium      2  1.4
  Medium High      Low         1  0.7
  Low    High      Low         1  0.7

```

Figure 29: Placement Path

```

=== TOP PLACEMENT PATHS: CSE (total placed: 404) ===
academic skill experience placed_count pct
    High   High      High          129 31.9
    High   High      Medium          86 21.3
    Medium High      High          80 19.8
    Medium High      Medium         54 13.4
    High   High      Low           27  6.7
    Medium High      Low           13  3.2
    Low    High      Medium          7  1.7
    Low    High      High            5  1.2
    Low    High      Low             3  0.7

=== TOP PLACEMENT PATHS: IT (total placed: 196) ===
academic skill experience placed_count pct
    High   High      High          71 36.2
    High   High      Medium         49 25.0
    Medium High      High          33 16.8
    Medium High      Medium         19  9.7
    High   High      Low           14  7.1
    Medium High      Low            6  3.1
    Low    High      High            2  1.0
    Low    High      Low             1  0.5
    Low    High      Medium          1  0.5

=== TOP PLACEMENT PATHS: ECE (total placed: 147) ===
academic skill experience placed_count pct
    High   High      High          42 28.6
    Medium High      High          32 21.8
    High   High      Medium         31 21.1
    Medium High      Medium         29 19.7
    High   High      Low            7  4.8
    Low    High      High            2  1.4
    Low    High      Medium          2  1.4
    Medium High      Low            1  0.7
    Low    High      Low             1  0.7

```

Figure 30: Placement Path

Graph Analysis: d3_100 Result

```
=====
STUDENT: d3_100 | Major: CSE | Status: NotPlaced
Profile: Academic=Medium | Skill=High | Experience=Medium
=====

LAYER 1 – GRAPH DISTANCE ANALYSIS:
-----
You are 1 hop(s) from the nearest placed CSE peer.
Nearest placed peer: d3_1035
5 placed CSE students within 1 hop(s).

LAYER 2 – PEER EVIDENCE (from graph):
-----
Among 404 placed CSE students:
→ 100% have Skill:High ✓ you have this
→ 60% have Academic:High ← MISSING
→ 53% have Experience:High ← MISSING

54 placed peers share your exact profile
(Acad:Medium, Skill:High, Exp:Medium) = 13% of placed CSE.

LAYER 3 – GRAPH RECOMMENDATION:
-----

Priority 1 – EXPERIENCE (lift: +0.107)
Upgrading Experience: Medium → High
connects you to +67 more placed CSE students.

Priority 2 – ACADEMIC (lift: -0.058)
Lift is NEGATIVE for CSE – do NOT prioritize Academic.

Priority 3 – SKILL (lift: +0.072)
Already optimal – maintain Skill:High.

— BOTTOM LINE —
One change (Experience: Medium → High) connects you to +67 more placed peers in CSE.
```

Figure 31: D3 100 Result

The first layer that the student is one hop away from the nearest placed CSE peer, which shows a direct SIMILAR_TO relationship exists between the two students, which is created when they share at least two attribute buckets (academic, skill, or experience). Additionally, there are five placed CSE students within one hop, indicating that the student's profile already overlaps with several successful peers.

The second layer shows that among the 404 placed CSE students (from Figure 8), 100% have Skill: High, which matches the student's current skill level. However, 60% of placed students have Academic: High and 53% have Experience: High, while the student currently has Academic: Medium and Experience: Medium, indicating that these attributes may be areas for improvement. The analysis also finds that 54 placed students share the same exact profile as the student, representing 13% of all placed CSE students, meaning that placement is still possible with the current profile but may not be optimal.

The third layer generates a graph-based recommendation by combining peer evidence with lift statistics calculated from the Task 0 dataset. Experience has the highest positive lift (+0.107), meaning that students with high experience are significantly more likely to be placed. Therefore, upgrading Experience from Medium to High would connect the student to **67 additional placed peers** in the graph, indicating a stronger alignment with successful profiles. In contrast, Academic lift is negative (-0.058), suggesting that improving academic performance does not significantly increase placement likelihood in this dataset. The student already has Skill: High, which is the most common attribute among placed students, so maintaining this level is recommended.

d3_1005 Result:

```
=====
STUDENT: d3_1005 | Major: CSE | Status: Placed
Profile: Academic=High | Skill=High | Experience=High
=====

LAYER 1 – GRAPH DISTANCE ANALYSIS:
-----
You are 1 hop(s) from the nearest placed CSE peer.
Nearest placed peer: d3_1011
5 placed CSE students within 1 hop(s).

LAYER 2 – PEER EVIDENCE (from graph):
-----
Among 404 placed CSE students:
→ 100% have Skill:High ✓ you have this
→ 60% have Academic:High ✓ you have this
→ 53% have Experience:High ✓ you have this

129 placed peers share your exact profile
(Acad:High, Skill:High, Exp:High) = 32% of placed CSE.

LAYER 3 – GRAPH RECOMMENDATION:
-----

Priority 1 – ACADEMIC (lift: -0.058)
  Already optimal – maintain Academic:High.

Priority 2 – SKILL (lift: +0.072)
  Already optimal – maintain Skill:High.

Priority 3 – EXPERIENCE (lift: +0.107)
  Already optimal – maintain Experience:High.

— BOTTOM LINE —
No positive-lift upgrade is recommended for this profile.
```

Figure 32: D3 1005 Analysis

This analysis evaluates the student **d3_1005**, whose profile is Academic: High, Skill: High, Experience: High, and who has already achieved a placement outcome.

In Layer 1 the result shows that the student is one hop away from another placed CSE peer, which shows a direct **SIMILAR_TO** relationship exists between the two students, which is created when they share at least two attribute buckets (academic, skill, or experience). Additionally, there are five placed CSE students within one hop, indicating that this student is strongly connected to other successful profiles in the network.

In Layer 2, among the 404 placed CSE students, 100% have Skill: High, 60% have Academic: High, and 53% have Experience: High. The student already **satisfies all three of these attributes**. The system also shows that 129 placed students share the exact same profile (High/High/High), representing approximately 32% of all placed CSE students. This indicates that the student's current profile closely matches the most common successful placement path.

In Layer 3, since the student already has High levels in all three categories, there is no attribute upgrade that would increase placement likelihood further. Therefore, the system recommends maintaining the current profile rather than making any changes.

3 Hop visualization for d3_100

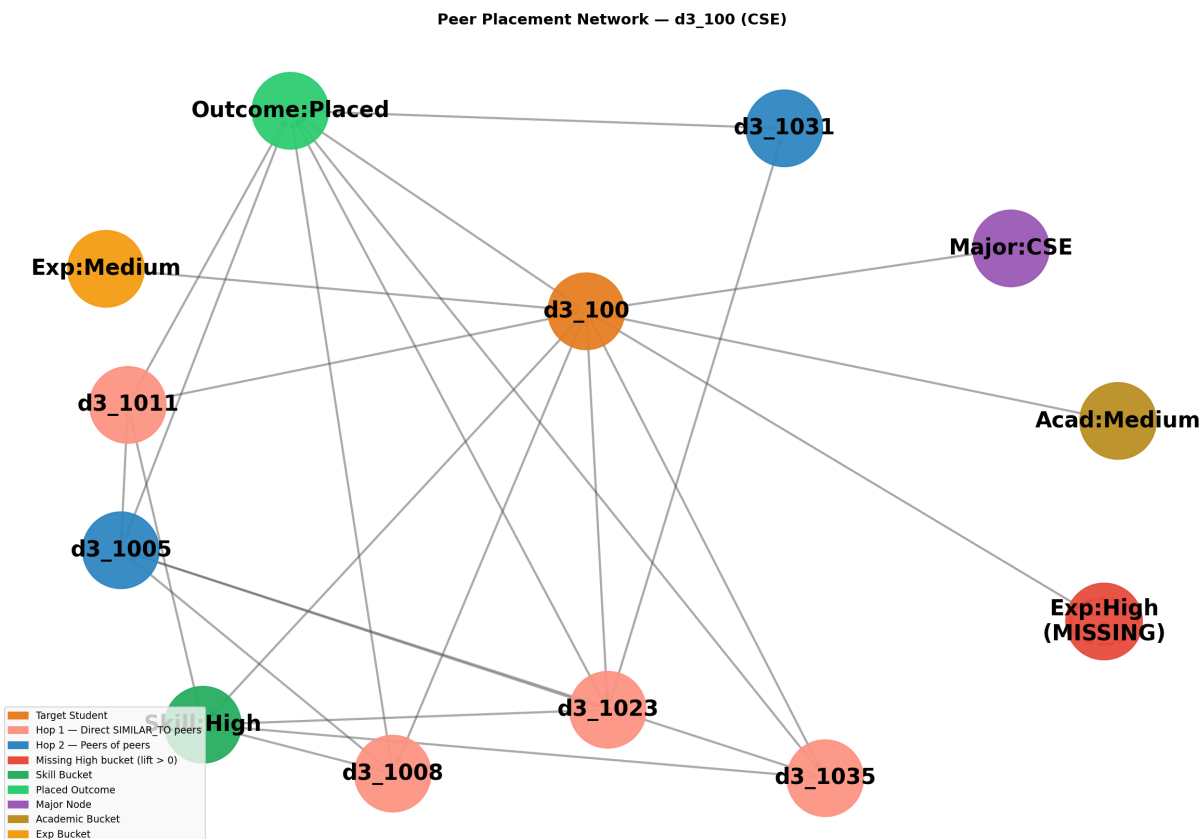


Figure 33: Peer network hop visualization

The figure visualizes the 3-hop neighborhood search around the student, showing direct peers (1-hop) and extended peer connections through additional similarity links. However, in this visualization only the first two hops are shown because most relevant peers already appear within two hops given the limited dataset and graph density.

The orange node represents the target student, while nearby nodes represent peers connected through shared attribute buckets or similarity relationships. Pink nodes represent direct similar peers (1-hop), and blue nodes represent peers reachable through an additional hop in the network. Green nodes indicate important attribute buckets shared among students, such as Skill:High, while the red node highlights a **missing attribute (Experience:High)** that has positive lift and may improve placement outcomes.

The visualization demonstrates that upgrading the student's experience bucket could connect the student to additional placed peers in the network, suggesting that experience is the most impactful improvement.

Limitations and Future Directions

Limitations

One main limitation is the data that is being used for this application. The datasets used in Task 0 come from a fixed historical dataset. This data does not give a very good overview of the job market overall. With the current data, the computations that are used are very limited in scope, under-representative of certain majors, so computations such as lift estimates could be statistically unreliable.

The resumes used in Task 1-4 (the 20 resumes and 3 improved versions, and 20 master resumes) are synthetic resumes generated. This causes category scores to cluster, which once again limits the meaningfulness of bucket distinctions in Task 1, and reduces the diversity of nearest-neighbor results in Task 4.

In Task 4, `placement_prediction` is computed, and not observed, thus not a ground-truth label. Task 5 is also only really computed around the CSE major as of right now. Would require more data surrounding other majors to incorporate into the feature.

Tasks are currently all operating independently, no live feedback loop that is connecting them. For example, when task 2 is run, it is not triggering task 1, but rather just pulling the results produced by Task 1 to operate.

Future Directions

One of the most important future improvements to the application is to use real resume ingestion into the tasks to replace generated resumes. This will allow the system to produce more diverse score distributions, which will also make bucket assignments and similarity searches more meaningful and insightful.

Another improvement that is looked into is the replacement of the current foundational data that is produced in Task 0. As more students use the system, task 0 data could incorporate more recent data surrounding job hunters, which can provide more accurate lift computations. Allowing the application to be more accurate over time.

It would also be beneficial to create a pipeline that will connect all the tasks together instead of producing them individually.